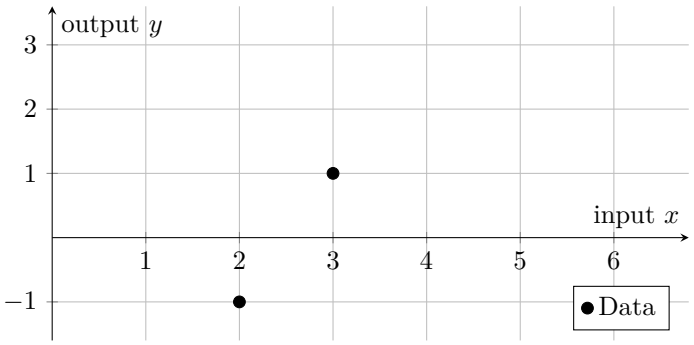
Linear regression

1.1 Linear regression

(a) Assume that you record a scalar input x and a scalar output y. First, you record $x_1 = 2, y_1 = -1$, and thereafter $x_2 = 3, y_2 = 1$. Assume a linear regression model $y = \theta_0 + \theta_1 x + \epsilon$ and learn the parameters with maximum likelihood $\widehat{\boldsymbol{\theta}}$ with the assumption $\epsilon \sim \mathcal{N}(0, \sigma_\epsilon^2)$. Use the model to predict the output for the test input $x_\star = 4$, and add the model to the plot below:



(b) Now, assume you have made a third observation $y_3 = 2$ for $x_3 = 4$ (is that what you predicted in (a)?). Update the parameters $\widehat{\boldsymbol{\theta}}$ to all 3 data samples, add the new model to the plot (together with the new data point) and find the prediction for $x_\star = 5$.

(c) Repeat (b), but this time using a model without intercept term, i.e., $y = \theta_1 x + \epsilon$.

(d) Repeat (b), but this time using Ridge Regression with $\gamma = 1$ instead.

(e) You realize that there are actually *two* output variables in the problem you are studying. In total, you have made the following observations:

sample	input x	first output y_1	second output y_2
(1)	2	-1	0
(2)	3	1	2
(3)	4	2	-1

You want to model this as a linear regression with multidimensional outputs (without regularization), i.e.,

$$y_1 = \theta_{01} + \theta_{11} x + \epsilon \tag{1.1}$$
$$y_2 = \theta_{02} + \theta_{12} x + \epsilon \tag{1.2}$$

By introducing, for the general case of p inputs and q outputs, the matrices

$$\underbrace{\begin{bmatrix} y_{11} & \cdots & y_{1q} \\ y_{21} & \cdots & y_{2q} \\ \vdots & & \vdots \\ y_{n1} & \cdots & y_{nq} \end{bmatrix}}_{\mathbf{Y}} = \underbrace{\begin{bmatrix} 1 & x_{11} & x_{12} & \cdots & x_{1p} \\ 1 & x_{21} & x_{22} & \cdots & x_{2p} \\ \vdots & \vdots & \vdots & & \vdots \\ 1 & x_{n1} & x_{n2} & \cdots & x_{np} \end{bmatrix}}_{\mathbf{X}} \underbrace{\begin{bmatrix} \theta_{01} & \theta_{02} & \cdots & \theta_{0q} \\ \theta_{11} & \theta_{12} & \cdots & \theta_{1q} \\ \theta_{21} & \theta_{22} & \cdots & \theta_{2q} \\ \vdots & \vdots & & \vdots \\ \theta_{p1} & \theta_{p2} & \cdots & \theta_{pq} \end{bmatrix}}_{\boldsymbol{\Theta}} + \boldsymbol{\epsilon}, \tag{1.3}$$

try to make an educated guess how the normal equations can be generalized to the multidimensional output case. (A more thorough derivation is found in problem 1.5). Use your findings to compute the least square solution $\widehat{\boldsymbol{\Theta}}$ to the problem now including both the first output y_1 and the second output y_2.

1.2 Nonlinear transformations of input variables

Assume our data set only contains two variables y and v. What are the transformed inputs/features if we want to learn a linear regression model on the form?

(a) $y = \theta_0 + \theta_1 v + \epsilon$

(b) $y = \theta_0 + \theta_1 v + \theta_2 v^2 + \theta_3 v^3 + \theta_4 v^4 + \epsilon$

(c) $y = \theta_0 + \theta_1 v + \theta_2 \cos(v) + \theta_3 \sin(v) + \epsilon$

(d) $y = \theta_0 + c\sin(v + \phi) + \epsilon$ (neither c nor ϕ are known) *Hint:* $\sin(v + \phi) = \cos(\phi)\sin(v) + \sin(\phi)\cos(v)$.

(e) $y = \theta_0 + \max(\theta_1 v, \theta_2 v^2) + \epsilon$?

1.3 Deriving least squares from maximum likelihood

We assume that each ϵ in the linear regression model is independently distributed as

$$\epsilon \sim \mathcal{N}\left(0,\, \sigma_\epsilon^2\right). \tag{1.4a}$$

We are interested in finding the maximum likelihood estimate of $\boldsymbol{\theta}$

$$\widehat{\boldsymbol{\theta}} = \arg\max_{\boldsymbol{\theta}} p(\mathbf{y} \,|\, \mathbf{X}; \boldsymbol{\theta}) \qquad \text{where} \qquad p(\mathbf{y} \,|\, \mathbf{X}; \boldsymbol{\theta}) = \prod_{i=1}^{n} p(y_i \,|\, \mathbf{x}_i; \boldsymbol{\theta}). \tag{1.4b}$$

(The factorization can be done since we assume ϵ to be independent for each data point i.) Show that this implies the least squares problem

$$\widehat{\boldsymbol{\theta}} = \arg\min_{\boldsymbol{\theta}} \left\{ \sum_{i=1}^{n} \left(\theta_0 + \theta_1 x_{i1} + \cdots + \theta_p x_{ip} - y_i\right)^2 \right\}. \tag{1.5}$$

Hint: You only need to show that this implies the least squares problem, not solve the least squares problem (i.e., derive the normal equations). If z is distributed as a Gaussian distribution with mean m and variance σ^2, its probability density is

$$\mathcal{N}\left(z \,|\, m,\, \sigma^2\right) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{1}{2\sigma^2}\left(z - m\right)^2\right).$$

1.4 The inverse of $\mathbf{X}^\mathsf{T}\mathbf{X}$

(a) Prove that $\mathbf{X}^\mathsf{T}\mathbf{X}$ is a positive definite matrix if and only if the columns of $\mathbf{X}$ are linearly independent.
Hint: A matrix $\mathbf{A}$ is by definition positive definite if and only if $v^\mathsf{T}\mathbf{A}v > 0$ for all column vectors $v \neq 0$.

(b) Use the result from (a) to prove that the unique solution $\widehat{\theta}$ from the normal equations exists if and only if the columns of $\mathbf{X}$ are linearly independent.

(c) Interpret (b) in terms of what properties your data matrix $\mathbf{X}$ needs to have. At least how many data samples do you need to have? What intuitive explanation can you give in the case there is no unique solution $\widehat{\boldsymbol{\theta}}$ to the least squares problem?

1.5 Optimum of multivariable least square

Consider again least squares with multivariable output. With the notation $\mathbf{y}_i = [y_{i1}\ y_{i2} \ldots y_{iq}]^\mathsf{T}$ and $\mathbf{x}_i = [1\ x_{i1}\ x_{i2} \ldots x_{ip}]^\mathsf{T}$. Prove that (1.11) (your finding on problem 1.1(e)) is a stationary point of the multivariable least squares criterion

$$\sum_{i=1}^{n} (\mathbf{y}_i - \boldsymbol{\Theta}^\mathsf{T}\mathbf{x}_i)^\mathsf{T}(\mathbf{y}_i - \boldsymbol{\Theta}^\mathsf{T}\mathbf{x}_i), \tag{1.6}$$

if you assume that $\mathbf{X}^\mathsf{T}\mathbf{X}$ is invertible. Three useful equations are

(i) $\frac{\partial}{\partial \mathbf{M}}\left(\mathbf{v}^\mathsf{T}\mathbf{M}\mathbf{M}^\mathsf{T}\mathbf{v}\right) = 2\mathbf{v}\mathbf{v}^\mathsf{T}\mathbf{M}$ for any column vector $\mathbf{v}$ of appropriate size.

(ii) $\frac{\partial}{\partial \mathbf{M}}\left(\mathbf{v}^\mathsf{T}\mathbf{M}\mathbf{u}\right) = \mathbf{v}\mathbf{u}^\mathsf{T}$ for any column vectors $\mathbf{u}, \mathbf{v}$ of appropriate sizes.

(iii) $\sum_{i=1}^{n} \mathbf{x}_i\mathbf{x}_i^\mathsf{T} = \mathbf{X}^\mathsf{T}\mathbf{X}$ and $\sum_{i=1}^{n} \mathbf{x}_i\mathbf{y}_i^\mathsf{T} = \mathbf{X}^\mathsf{T}\mathbf{Y}$.

Solutions

1.1 (a) Write the problem as $\underbrace{\begin{bmatrix} y_1 \\ y_2 \end{bmatrix}}_{\mathbf{y}} = \underbrace{\begin{bmatrix} 1 & x_1 \\ 1 & x_2 \end{bmatrix}}_{\mathbf{X}} \underbrace{\begin{bmatrix} \theta_0 \\ \theta_1 \end{bmatrix}}_{\mathbf{\Theta}} + \boldsymbol{\epsilon}$. The maximum likelihood solution to this problem is equivalent to the least square solution given by $\mathbf{X}^\mathsf{T}\mathbf{X}\widehat{\mathbf{\Theta}} = \mathbf{X}^\mathsf{T}\mathbf{y}$. We thus solve it (using, e.g., Gauss elimination),

$$\begin{bmatrix} 1 & 2 \\ 1 & 3 \end{bmatrix}^\mathsf{T} \begin{bmatrix} 1 & 2 \\ 1 & 3 \end{bmatrix} \widehat{\mathbf{\Theta}} = \begin{bmatrix} 1 & 2 \\ 1 & 3 \end{bmatrix}^\mathsf{T} \begin{bmatrix} -1 \\ 1 \end{bmatrix} \Rightarrow \begin{bmatrix} 2 & 5 \\ 5 & 13 \end{bmatrix} \widehat{\mathbf{\Theta}} = \begin{bmatrix} 0 \\ 1 \end{bmatrix} \Rightarrow \widehat{\mathbf{\Theta}} = \begin{bmatrix} -5 \\ 2 \end{bmatrix}. \tag{1.7}$$

The prediction for $x_\star = 4$ becomes $\widehat{y}_\star = \widehat{\theta}_0 + \widehat{\theta}_1 x_\star = 3$.

(b) Again, the solution is given by the normal equations $\mathbf{X}^\mathsf{T}\mathbf{X}\widehat{\mathbf{\Theta}} = \mathbf{X}^\mathsf{T}\mathbf{y}$

$$(\mathbf{X}^\mathsf{T}\mathbf{X})\widehat{\mathbf{\Theta}} = \mathbf{X}^\mathsf{T}\mathbf{y} \Rightarrow \left(\begin{bmatrix} 1 & 1 & 1 \\ 2 & 3 & 4 \end{bmatrix} \begin{bmatrix} 1 & 2 \\ 1 & 3 \\ 1 & 4 \end{bmatrix} \right) \widehat{\mathbf{\Theta}} = \begin{bmatrix} 1 & 1 & 1 \\ 2 & 3 & 4 \end{bmatrix} \begin{bmatrix} -1 \\ 1 \\ 2 \end{bmatrix} \Rightarrow \cdots \Rightarrow \widehat{\mathbf{\Theta}} = \frac{1}{6} \begin{bmatrix} -23 \\ 9 \end{bmatrix}. \tag{1.8}$$

The prediction for $x_\star = 5$ is hence $\widehat{y}_\star = \widehat{\theta}_0 + \widehat{\theta}_1 x_\star = \frac{11}{3} \approx 3.67$.

(c) With no intercept term, we get another $\mathbf{X}$ matrix, $\begin{bmatrix} 2 \\ 3 \\ 4 \end{bmatrix}$, and hence

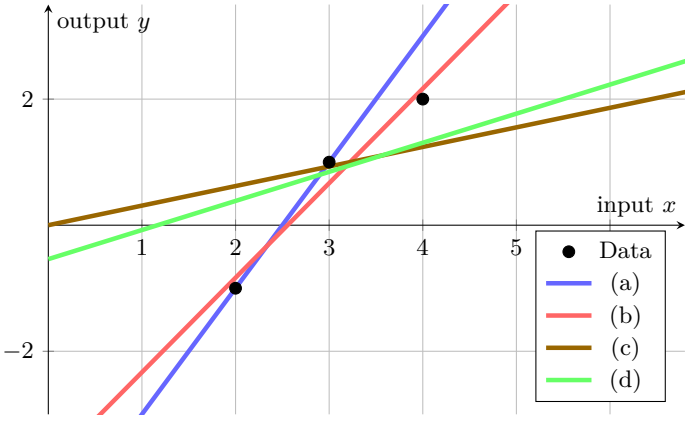
$$\mathbf{X}^\mathsf{T}\mathbf{X}\widehat{\mathbf{\Theta}} = \mathbf{X}^\mathsf{T}\mathbf{y} \Rightarrow \begin{bmatrix} 2 & 3 & 4 \end{bmatrix} \begin{bmatrix} 2 \\ 3 \\ 4 \end{bmatrix} \widehat{\mathbf{\Theta}} = \begin{bmatrix} 2 & 3 & 4 \end{bmatrix} \begin{bmatrix} -1 \\ 1 \\ 2 \end{bmatrix} \Rightarrow \cdots \Rightarrow \widehat{\mathbf{\Theta}} = \frac{9}{29}, \tag{1.9}$$

with prediction $\widehat{y}_\star = \widehat{\theta}_1 x_\star = \frac{45}{29} \approx 1.55$ for $x_\star = 5$.

(d) We now have to use the solution to the Ridge Regression problem instead, $(\mathbf{X}^\mathsf{T}\mathbf{X} + I_2)\widehat{\boldsymbol{\theta}} = \mathbf{X}^\mathsf{T}\mathbf{y}$,

$$(\mathbf{X}^\mathsf{T}\mathbf{X} + I_2)\widehat{\mathbf{\Theta}} = \mathbf{X}^\mathsf{T}\mathbf{y} \Rightarrow \left(\begin{bmatrix} 1 & 1 & 1 \\ 2 & 3 & 4 \end{bmatrix} \begin{bmatrix} 1 & 2 \\ 1 & 3 \\ 1 & 4 \end{bmatrix} + \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \right) \widehat{\mathbf{\Theta}} = \begin{bmatrix} 1 & 1 & 1 \\ 2 & 3 & 4 \end{bmatrix} \begin{bmatrix} -1 \\ 1 \\ 2 \end{bmatrix} \Rightarrow \cdots \Rightarrow \widehat{\mathbf{\Theta}} = \frac{1}{39} \begin{bmatrix} -21 \\ 18 \end{bmatrix}. \tag{1.10}$$

The prediction for $x_\star = 5$ is hence $\widehat{y}_\star = \widehat{\theta}_0 + \widehat{\theta}_1 x_\star = \frac{69}{39} \approx 1.77$.



(e) The extension of the normal equations are

$$\mathbf{X}^\mathsf{T}\mathbf{X}\widehat{\mathbf{\Theta}} = \mathbf{X}^\mathsf{T}\mathbf{Y}. \tag{1.11}$$

Note that this is equivalent to making a separate least square computation for each column in $\mathbf{Y}$.

$$\mathbf{X}^\mathsf{T}\mathbf{X}\widehat{\mathbf{\Theta}} = \mathbf{X}^\mathsf{T}\mathbf{Y} \Rightarrow \left(\begin{bmatrix} 1 & 1 & 1 \\ 2 & 3 & 4 \end{bmatrix} \begin{bmatrix} 1 & 2 \\ 1 & 3 \\ 1 & 4 \end{bmatrix} \right) \widehat{\mathbf{\Theta}} = \begin{bmatrix} 1 & 1 & 1 \\ 2 & 3 & 4 \end{bmatrix} \begin{bmatrix} -1 & 0 \\ 1 & 2 \\ 2 & -1 \end{bmatrix} \Rightarrow \cdots \Rightarrow \widehat{\mathbf{\Theta}} = \frac{1}{6} \begin{bmatrix} -23 & 11 \\ 9 & -3 \end{bmatrix}. \tag{1.12}$$

Note that the first column is identical to (b).

1.2 (a) No extra inputs are needed, $x_1 = v$.

(b) $x_1 = v, x_2 = v^2, x_3 = v^3, x_4 = v^4$.

(c) $x_1 = v, x_2 = \cos(v), x_3 = \sin(v)$.

(d) We can write the model as $y = \theta_0 + \underbrace{c\cos(\phi)}_{\theta_1}\sin(v) + \underbrace{c\sin(\phi)}_{\theta_2}\cos(v) + \epsilon$, hence $x_1 = \sin(v), x_2 = \cos(v)$.

(e) The model is not possible to write as a linear regression model.

1.3 According to the exercise description, the data distribution has the form

$$\begin{aligned} p(\mathbf{y} \,|\, \mathbf{X}; \boldsymbol{\theta}) &= \prod_{i=1}^{n} p(y_i \,|\, \mathbf{x}_i; \boldsymbol{\theta}) \\ &= \prod_{i=1}^{n} \mathcal{N}\Big(\underbrace{\theta_0 + \theta_1 x_{i1} + \cdots + \theta_p x_{ip} - y_i}_{\epsilon} \,|\, 0,\, \sigma_\epsilon^2 \Big) \\ &\propto \prod_{i=1}^{n} e^{-\frac{1}{2\sigma_\epsilon^2}(\theta_0 + \theta_1 x_{i1} + \cdots + \theta_p x_{ip} - y_i)^2} \\ &= \exp\left(\sum_{i=1}^{n} -\frac{1}{2\sigma_\epsilon^2} \left(\theta_0 + \theta_1 x_{i1} + \cdots + \theta_p x_{ip} - y_i\right)^2 \right) \end{aligned} \tag{1.13}$$

Since log is a monotonically increasing function, we can write

$$\begin{aligned} \widehat{\theta} &= \arg\max_{\theta} p(\mathbf{y} \,|\, \mathbf{X}; \boldsymbol{\theta}) \\ &= \arg\max_{\theta} \log p(\mathbf{y} \,|\, \mathbf{X}; \boldsymbol{\theta}) \\ &= \arg\max_{\theta} \left\{ \sum_{i=1}^{n} -\frac{1}{2\sigma^2} \left(\theta_0 + \theta_1 x_{i1} + \cdots + \theta_p x_{ip} - y_i\right)^2 \right\} \\ &= \arg\min_{\theta} \left\{ \sum_{i=1}^{n} \left(\theta_0 + \theta_1 x_{i1} + \cdots + \theta_p x_{ip} - y_i\right)^2 \right\} \end{aligned} \tag{1.14}$$

1.4 (a) A matrix $\mathbf{A}$ is by definition positive definite if and only if $\mathbf{v}^\mathsf{T}\mathbf{A}\mathbf{v} > 0$ for all column vectors $v \neq 0$. We can write $\mathbf{v}^\mathsf{T}\mathbf{X}^\mathsf{T}\mathbf{X}\mathbf{v} = (\mathbf{X}\mathbf{v})^\mathsf{T}(\mathbf{X}\mathbf{v}) = \|\mathbf{X}\mathbf{v}\|^2$. Then $\|\mathbf{X}\mathbf{v}\|^2 = 0 \Leftrightarrow \mathbf{X}\mathbf{v} = 0$, which means that $\mathbf{v}$ is in the null space of $\mathbf{X}$. If $\mathbf{v} \neq 0$, it would mean that $\mathbf{X}$ would have linearly dependent columns, and the claim follows.

(b) The normal equations are only defined if the inverse of $\mathbf{X}^\mathsf{T}\mathbf{X}$ exists, which it does if and only if $\mathbf{X}^\mathsf{T}\mathbf{X}$ is positive definite (by construction, it cannot be negative definite or indefinite). From (a), this is equivalent to $\mathbf{X}$ having linearly independent columns.

(c) A necessary condition for $\mathbf{X}$ having linearly independent columns means that there has to be at least as many data samples n (i.e., rows), as the number of inputs p (i.e., columns). If the columns of $\mathbf{X}$ are not linearly independent, it means that there is no information in the data that can tell the different inputs apart (e.g., if the very same input is used for all recorded data points). *It does, however,* ***not*** *mean that a solution does not exist. The least squares problem has in such cases an infinite number of equally good solutions.*

1.5 Differentiating (1.6) yields with respect to $\boldsymbol{\Theta}$

$$\begin{aligned} \frac{\partial}{\partial \boldsymbol{\Theta}} \sum_{i=1}^{n} (\mathbf{y}_i - \boldsymbol{\Theta}^\mathsf{T}\mathbf{x}_i)^\mathsf{T}(\mathbf{y}_i - \boldsymbol{\Theta}^\mathsf{T}\mathbf{x}_i) &= \frac{\partial}{\partial \boldsymbol{\Theta}} \sum_{i=1}^{n} \left(\mathbf{y}_i^\mathsf{T}\mathbf{y}_i - 2\mathbf{x}_i^\mathsf{T}\boldsymbol{\Theta}\mathbf{y}_i + \mathbf{x}_i^\mathsf{T}\boldsymbol{\Theta}\boldsymbol{\Theta}^\mathsf{T}\mathbf{x}_i \right) = /\text{(i) \& (ii)}/ = \\ \sum_{i=1}^{n} \left(0 - 2\mathbf{x}_i\mathbf{y}_i^\mathsf{T} + 2\mathbf{x}_i\mathbf{x}_i^\mathsf{T}\boldsymbol{\Theta}\right) &= 2\left(\sum_{i=1}^{n} \mathbf{x}_i\mathbf{x}_i^\mathsf{T}\right)\boldsymbol{\Theta} - 2\left(\sum_{i=1}^{n} \mathbf{x}_i\mathbf{y}_i^\mathsf{T}\right) = /\text{(iii)}/ = 2(\mathbf{X}^\mathsf{T}\mathbf{X}\boldsymbol{\Theta} - \mathbf{X}^\mathsf{T}\mathbf{Y}) \end{aligned} \tag{1.15}$$

Inserting $\widehat{\boldsymbol{\Theta}} = (\mathbf{X}^\mathsf{T}\mathbf{X})^{-1}\mathbf{X}^\mathsf{T}\mathbf{Y}$ (since we assumed $\mathbf{X}^\mathsf{T}\mathbf{X}$ to be invertible) into (1.15) gives

$$2(\mathbf{X}^\mathsf{T}\mathbf{X}\widehat{\boldsymbol{\Theta}} - \mathbf{X}^\mathsf{T}\mathbf{Y}) = 2\left((\mathbf{X}^\mathsf{T}\mathbf{X})(\mathbf{X}^\mathsf{T}\mathbf{X})^{-1}\mathbf{X}^\mathsf{T}\mathbf{Y} - \mathbf{X}^\mathsf{T}\mathbf{Y}\right) = \mathbf{0}. \tag{1.16}$$

Classification

3.1 k nearest neighbor

The table below is the training data set with $n = 6$ observations of a 3-dimensional categorical input $\mathbf{x} = [x_1\ x_2\ x_3]^\mathsf{T}$ and 1 categorical output y (the color green or red).

i	$\mathbf{x}$			y
1	0	3	0	Red
2	2	0	0	Red
3	0	1	3	Red
4	0	1	2	Green
5	−1	0	1	Green
6	1	1	1	Red

(a) Compute the Euclidean distance between each observation in the training data, and the test point $\mathbf{x}_\star = [0\ 0\ 0]^\mathsf{T}$.

(b) What is our prediction for the test point $\mathbf{x}_\star$, if we use k-NN with $k = 1$?

(c) What is our prediction for the test point $\mathbf{x}_\star$, if we use k-NN with $k = 3$?

3.2 Logistic regression

Suppose we collect data from a group of students in a Machine learning class with variables x_1 = hours studied, x_2 = grade point average, and y = a binary output if that student received grade 5 ($y = 1$) or not ($y = -1$). We learn a logistic regression model

$$p(y = 1 \,|\, \mathbf{x}) = \frac{e^{\theta_0+\theta_1 x_1+\theta_2 x_2}}{1 + e^{\theta_0+\theta_1 x_1+\theta_2 x_2}} \tag{3.1}$$

with parameters $\widehat{\theta}_0 = -6$, $\widehat{\theta}_1 = 0.05$, $\widehat{\theta}_2 = 1$.

(a) Estimate the probability according to the logistic regression model that a student who studies for 40 h and has the grade point average of 3.5 gets a 5 in the Machine learning class.

(b) According to the logistic regression model, how many hours would the student in part (a) need to study to have 50% chance of getting a 5 in the class?

3.3 Difference between LDA and QDA

We now examine the differences between LDA and QDA.

(a) If the optimal boundary is linear, do we expect LDA or QDA to perform better on the training set? What do we expect on the test set?

(b) If the optimal decision boundary is nonlinear, do we expect LDA or QDA to perform better on the training set? What do we expect on the test set?

(c) In general, as the sample size n increases, do we expect the test error rate of QDA relative to LDA to increase, decrease or be unchanged? Why?

(d) True or false: Even if the optimal decision boundary for a given problem is linear, we will probably achieve a smaller test error rate using QDA rather than LDA because QDA is flexible enough to model a linear decision boundary. Justify your answer.

3.4 The optimal classifier

Suppose you work at a clinic in a field mission, and want to predict whether a patient has a particular (potentially deadly) disease or not. You do have a limited supply of an effective drug, which however has severe side effects. Due to unfortunate circumstances, the only diagnosis tool you have access to is a clinical thermometer, with which you can measure the body temperature of the patient. From previous studies made on the disease, you know the following:

- the distribution of body temperatures in infected patients is approximately Gaussian distributed with mean 38.5 °C and standard deviation 1 °C.
- the distribution of body temperatures in patients not infected by the disease (either healthy or infected by other diseases) is approximately Gaussian with mean 37.5 °C and standard deviation $\sqrt{0.5}$ °C.
- the prevalence of the disease is 5% (i.e., 5% of the population is infected).

The body temperatures of three patients are as follows: patient A 38.5 °C, patient B 39.2 °C, and patient C 40.1 °C.

(a) What is the probability that patient A, B and C are infected by the disease, respectively?
Hint: Use

$$p(y \mid \mathbf{x}) = \frac{p(\mathbf{x} \mid y)p(y)}{\sum_{m=1}^{M} p(\mathbf{x} \mid m)p(m)},$$

where $p(y)$ is the prior probability of class y, and $p(\mathbf{x} \mid y)$ the probability density of $\mathbf{x}$ for an observation from class y. (Bayes' theorem)

(b) Which prediction should you make for each patient, in order to make on average as few misclassifications as possible? (Hint: The most probable one. (Bayes' classifier))

(c) Argue why another performance metric other than the standard accuracy ('on average as few misclassifications as possible') should be considered for this problem. How would that affect your decisions in (b)?

3.5 Error rates

Suppose that we take a data set, divide it into equally-sized training and test sets, and then try out two different classification procedures. First we use logistic regression and get an error rate of 20% on the training data and 30% on the test data. Next, we use 1-nearest neighbors (i.e. k-NN with $k = 1$) and get an average error rate (averaged over both test and training data sets) of 18%. Based on these results, which method should we prefer to use for classification of new observations? Why?

3.6 Quadratic Discriminant Analysis

Consider a classification problem with the input $\mathbf{x} \in \mathbb{R}^p$ and output $y \in \{1, \dots, K\}$. Consider also Bayes' classifier

$$\widehat{y} = \underset{k=\{1,\dots,K\}}{\arg\max}\ p(y = k \mid \mathbf{x}), \qquad \text{where} \qquad p(y \mid \mathbf{x}) = \frac{p(\mathbf{x} \mid y)p(y)}{\sum_{k=1}^{K} p(\mathbf{x} \mid k)p(k)}. \tag{3.2}$$

In Quadratic Discriminant Analysis (QDA), we assume that $p(\mathbf{x} \mid y)$ is a multivariate Gaussian density with mean $\boldsymbol{\mu}_k$ and covariance $\boldsymbol{\Sigma}_k$

$$p(\mathbf{x} \mid y = k) = \mathcal{N}(\mathbf{x} \mid \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k) = \frac{1}{(2\pi)^{p/2}|\boldsymbol{\Sigma}_k|^{1/2}} e^{-\frac{1}{2}(\mathbf{x}-\boldsymbol{\mu}_k)^\top \boldsymbol{\Sigma}_k^{-1}(\mathbf{x}-\boldsymbol{\mu}_k)} \tag{3.3a}$$

$$p(y = k) = \pi_k \tag{3.3b}$$

(each with a different $\boldsymbol{\mu}_k$, $\boldsymbol{\Sigma}_k$ and π_k).

(a) Show that if one makes the assumption (3.3), then Bayes' classifier becomes

$$\widehat{y} = \underset{k=\{1,\dots,K\}}{\arg\max}\ \delta_k(\mathbf{x}), \quad \text{where} \quad \delta_k(\mathbf{x}) = -\frac{1}{2}\mathbf{x}^\top \boldsymbol{\Sigma}_k^{-1}\mathbf{x} + \mathbf{x}^\top \boldsymbol{\Sigma}_k^{-1}\boldsymbol{\mu}_k - \frac{1}{2}\boldsymbol{\mu}_k^\top \boldsymbol{\Sigma}_k^{-1}\boldsymbol{\mu}_k - \frac{1}{2}\log|\boldsymbol{\Sigma}_k| + \log \pi_k. \tag{3.4}$$

This is QDA, and $\delta_k(\mathbf{x})$ is called the discriminant function.
Hint: In lecture 4, an equivalent derivation for LDA was made, assuming $\boldsymbol{\Sigma}_k = \boldsymbol{\Sigma}$ is constant for all k. Look on that one and extend it accordingly for QDA by relaxing this assumption.

(b) Consider two classes k and l. Show that the decision boundary between these two classes is given by a quadratic function.

3.7 Curse of dimensionality

For large number of inputs p, some methods, such as the nonparametric k-NN, may perform bad due to the large dimensionality p of the input space. The problem is that the concept of 'near' or 'close' is very much depending on the number of dimensions p, and is commonly referred to as *the curse of dimensionality*. To investigate this, we will now consider an alternative version of the k-NN method, considering all neighbors within a fixed hypercube (instead of the k nearest) for making the decision.

(a) Suppose that $p = 1$, and that the inputs $\mathbf{x}$ are uniformly distributed on $[0, 1]$. We decide to consider all observations with an input within a ± 0.05 interval (as an alternative to using the k nearest observed inputs in the k-NN method) when making predictions. We now want to predict a test observation with input $X = 0.3$. On average, what fraction of all training observations will be used in making the prediction?

(b) Now consider the corresponding situation for $p = 2$: The inputs are uniformly distributed on $[0, 1] \times [0, 1]$, and for making predictions we use all training observations within ± 0.05 in each dimension. On average, what fraction of all training observations will we use when making a prediction for a test observation with input $\mathbf{x}_\star = [0.3\ 0.6]^\mathsf{T}$?

(c) In general, what fraction of all training observations will be used in predictions if there are p dimensions? As before, all inputs are uniformly distributed on $[0, 1]^p$ and for prediction we consider the training observations within ± 0.05 for each dimension. You may ignore the boundary effects if the test input is within 0.05 from the borders 0 or 1.

(d) Based on your answers to (a)-(c), argue why the prediction performance of k-NN might deteriorate for large p.

(e) If the inputs are distributed as in (c), and we want to make predictions using 10% of the training data inputs, which length should the side of a symmetric hypercube have, that covers on average 10% of the inputs?

Solutions

3.1 (a) The Euclidean distances are in the rightmost column below

i	$\mathbf{x}$			y	distance $\|\mathbf{x}-\mathbf{x}_\star\|$
1	0	3	0	Red	3
2	2	0	0	Red	2
3	0	1	3	Red	$\sqrt{10} \approx 3.2$
4	0	1	2	Green	$\sqrt{5} \approx 2.2$
5	-1	0	1	Green	$\sqrt{2} \approx 1.4$
6	1	1	1	Red	$\sqrt{3} \approx 1.7$

(b) k-NN with $k = 1$, the closest observation, i.e., observation 5, is selected as prediction. Thus, the prediction is green.

(c) The 3 closest observation are observation 5, 6, and 2. Thus, the prediction is red.

3.2 (a) The probability of getting a 5 using the parameters $\widehat{\theta}_0 = -6$, $\widehat{\theta}_1 = 0.05$ is

$$p(y = 1 \,|\, \mathbf{x}) = \frac{e^{\widehat{\theta}_0 + \widehat{\theta}_1 x_1 + \widehat{\theta}_2 x_2}}{1 + e^{\widehat{\theta}_0 + \widehat{\theta}_1 x_1 + \widehat{\theta}_2 x_2}} = \frac{e^{-6 + 0.051 x_1 + x_2}}{1 + e^{-6 + 0.05 x_1 + 1 x_2}} \tag{3.5}$$

Now, with $x_1 = 40$ and $x_2 = 3.5$,

$$p(y = 1 \,|\, \mathbf{x}) = \frac{e^{-6 + 0.05 \,\cdot\, 40 + 1 \,\cdot\, 3.5}}{1 + e^{-6 + 0.05 \,\cdot\, 40 + 1 \,\cdot\, 3.5}} = \frac{e^{-0.5}}{1 + e^{-0.5}} \tag{3.6}$$

$$= \frac{1}{1 + e^{0.5}} \approx 38\%. \tag{3.7}$$

(b) Set $p(y = 1 \,|\, \mathbf{x}) = 0.5$ and $x_2 = 3.5$. This gives

$$0.5 = \frac{e^{-6 + 0.05 x_1 + 3.5}}{1 + e^{-6 + 0.05 x_1 + 3.5}} = \frac{1}{e^{2.5 - 0.05 x_1} + 1} \Rightarrow \tag{3.8}$$

$$0.5(1 + e^{2.5 - 0.05 x_1}) = 1 \Rightarrow \tag{3.9}$$

$$e^{2.5 - 0.05 x_1} = \frac{1}{0.5} - 1 = 1 \Rightarrow \tag{3.10}$$

$$2.5 - 0.05 x_1 = \log(1) = 0 \Rightarrow \tag{3.11}$$

$$x_1 = \frac{2.5}{0.05} = 50 \text{ h}. \tag{3.12}$$

3.3 (a) We can always expect QDA to perform better than LDA on the *training* set because it is more flexible and is capable of fitting the training data better. If the optimal decision boundary is linear, we expect LDA to perform better on *test* data because it does not overfit.

(b) If the optimal decision boundary is nonlinear, we expect QDA to be able to perform better also on the test sets.

(c) In general we expect the test error rate to improve with QDA relative to LDA as the sample size n increases, since QDA is more flexible and will therefore be able to be closer to the optimal decision boundary. (However, for small n, it may overfit to the training data.)

(d) False. With few data points n, the QDA is likely to overfit (yielding a higher test error rate than LDA), which the LDA cannot if the Bayes decision boundary also is linear.

3.4 (a) We have the output y describing the patient status as $\{\text{infected}, \text{healthy}\}$, and the input $\mathbf{x}$ being the body temperature:

- $p(\mathbf{x} \,|\, y = \text{infected}) = \mathcal{N}(x \,|\, 38.5, 1)$
- $p(\mathbf{x} \,|\, y = \text{healthy}) = \mathcal{N}(x \,|\, 37.5, 0.5)$
- $p(\text{infected}) = 0.05$, and hence $p(\text{healthy}) = 0.95$

Inserting the expressions and patients temperatures into Bayes' theorem, we get

$$\begin{aligned} p(\text{Patient A is infected}) &= p(y = \text{infected} \,|\, \mathbf{x} = 38.5) \\ &= \frac{p(\mathbf{x} = 38.5 \,|\, y = \text{infected}) p(y = \text{infected})}{p(\mathbf{x} = 38.5 \,|\, y = \text{infected}) p(y = \text{infected}) + p(\mathbf{x} = 38.5 \,|\, y = \text{healthy}) p(y = \text{healthy})} \\ &= \frac{\mathcal{N}(38.5 \,|\, 38.5, 1) \cdot 0.05}{\mathcal{N}(38.5 \,|\, 38.5, 1) \cdot 0.05 + \mathcal{N}(38.5 \,|\, 37.5, 0.5) \cdot 0.95} \\ &\approx 0.09. \end{aligned}$$

Similarly, we obtain

$$\begin{aligned} p(\text{patient B is infected}) &= p(y = \text{infected} \,|\, \mathbf{x} = 39.2) \approx 0.34, \\ p(\text{patient C is infected}) &= p(y = \text{infected} \,|\, \mathbf{x} = 40.1) \approx 0.90. \end{aligned}$$

(b) Predicting the most likely class minimizes the average number of misclassifications. For this problem, it gives the following predictions: patient A healthy, patient B healthy, patient C infected.

(c) Since the disease is deadly, there is an asymmetry in the problem. The consequences of falsely classifying an infected patient as healthy is probably worse than falsely classifying a healthy patient as infected (despite the side effects of the drug). A classifier designed with this asymmetry in mind would probably also predict patient B, and perhaps also A, as infected.

A useful tool for such a design could be the confusion matrix.

3.5 Logistic regression has a training error rate of $P_{\text{training}} = 20\%$ and test error rate of $P_{\text{test}} = 30\%$. k-NN ($k = 1$): average error rate of $\frac{P_{\text{training}} + P_{\text{test}}}{2} = 18\%$.

However, for k-NN with k=1, the training error rate is $P_{\text{training}} = 0\%$ because for any training observation, its nearest neighbor will be the response itself. So, k-NN has a test error rate of $P_{\text{test}} = 36\%$. I would choose logistic regression because of its lower test error rate of 30%.

3.6 (a) Since the denominator in (3.2) does not depend on k, we get

$$\begin{aligned} \widehat{y} &= \underset{k=\{1,\ldots,K\}}{\arg\max}\ p(y = k|\mathbf{x}) \\ &= \underset{k=\{1,\ldots,K\}}{\arg\max}\ p(\mathbf{x} \,|\, k) p(k) \\ &= \underset{k=\{1,\ldots,K\}}{\arg\max}\ \log\Big(p(\mathbf{x} \,|\, k) p(k)\Big) \end{aligned}$$

Further, we see that

$$\begin{aligned}\log\Big(p(\mathbf{x}\,|\,k)p(k)\Big) &= \log\Big(p(k)\Big) + \log\Big(p(\mathbf{x}\,|\,k)\Big)\\ &= -\frac{1}{2}(\mathbf{x}-\boldsymbol{\mu}_k)^\mathsf{T}\boldsymbol{\Sigma}_k^{-1}(\mathbf{x}-\boldsymbol{\mu}_k) - \frac{1}{2}\log|\boldsymbol{\Sigma}_k| + \log(\pi_k) + \underbrace{\text{const.}}_{\text{independent of } k}\\ &= \underbrace{-\frac{1}{2}\mathbf{x}^\mathsf{T}\boldsymbol{\Sigma}_k^{-1}\mathbf{x} + \mathbf{x}^\mathsf{T}\boldsymbol{\Sigma}_k^{-1}\boldsymbol{\mu}_k - \frac{1}{2}\boldsymbol{\mu}_k^\mathsf{T}\boldsymbol{\Sigma}_k^{-1}\boldsymbol{\mu}_k - \frac{1}{2}\log|\boldsymbol{\Sigma}_k| + \log(\pi_k)}_{=\delta_k(\mathbf{x})} + \underbrace{\text{const.}}_{\text{independent of } k}\end{aligned}$$

where the classification problem can be written as

$$\widehat{y} = \underset{k=\{1,\dots,K\}}{\arg\max}\ \delta_k(\mathbf{x}). \tag{3.13}$$

(b) Compare two classes $y = k$ and $y = l$. The decision boundary between these two classes is given by

$$p(y = k|\mathbf{x}) = p(y = l|\mathbf{x}) \Rightarrow \delta_k(\mathbf{x}) - \delta_l(\mathbf{x}) = 0,$$

i.e., where the predicted probability for each of the two classes are equally high.

This gives

$$\begin{aligned}\delta_k(\mathbf{x}) - \delta(_l(\mathbf{x})) &= -\frac{1}{2}\mathbf{x}^\mathsf{T}\boldsymbol{\Sigma}_k^{-1}\mathbf{x} + \mathbf{x}^\mathsf{T}\boldsymbol{\Sigma}_k^{-1}\boldsymbol{\mu}_k - \frac{1}{2}\boldsymbol{\mu}_k^\mathsf{T}\boldsymbol{\Sigma}_k^{-1}\boldsymbol{\mu}_k - \frac{1}{2}\log|\boldsymbol{\Sigma}_k| + \log(\pi_k)\\ &\quad - \left(-\frac{1}{2}\mathbf{x}^\mathsf{T}\boldsymbol{\Sigma}_l^{-1}\mathbf{x} + \mathbf{x}^\mathsf{T}\boldsymbol{\Sigma}_l^{-1}\boldsymbol{\mu}_l - \frac{1}{2}\boldsymbol{\mu}_l^\mathsf{T}\boldsymbol{\Sigma}_l^{-1}\boldsymbol{\mu}_l - \frac{1}{2}\log|\boldsymbol{\Sigma}_l| + \log(\pi_l)\right)\\ &= -\frac{1}{2}\mathbf{x}^\mathsf{T}(\boldsymbol{\Sigma}_k^{-1} - \boldsymbol{\Sigma}_l^{-1})\mathbf{x} + \mathbf{x}^\mathsf{T}(\boldsymbol{\Sigma}_k^{-1}\boldsymbol{\mu}_k - \boldsymbol{\Sigma}_l^{-1}\boldsymbol{\mu}_l)\\ &\quad - \frac{1}{2}\boldsymbol{\mu}_k^\mathsf{T}\boldsymbol{\Sigma}_k^{-1}\boldsymbol{\mu}_k - \frac{1}{2}\log|\boldsymbol{\Sigma}_k| + \log(\pi_k) + \frac{1}{2}\boldsymbol{\mu}_l^\mathsf{T}\boldsymbol{\Sigma}_l^{-1}\boldsymbol{\mu}_l + \frac{1}{2}\log|\boldsymbol{\Sigma}_l| - \log(\pi_l),\end{aligned}$$

which is a quadratic function in $\mathbf{x}$ as long as $\boldsymbol{\Sigma}_k \neq \boldsymbol{\Sigma}_l$.

3.7 (a) All training observations with inputs on the interval $[0.25, 0.35]$ will be used for making the prediction. Since the inputs are uniformly distributed on the interval $[0, 1]$, will 10% on the average be in $[0.25, 0.35]$, and hence be used for the prediction.

(b) In this case, we will use all training observations with inputs in the square $[0.25, 0.35] \times [0.55, 0.65]$. The square covers 1% of $[0, 1] \times [0, 1]$, and hence will on average only 1% of the training observations be used in the predictions.

(c) The probability of an input to be inside the hypercube with dimensions $0.1 \times \cdots \times 0.1$ is 0.1 per dimension, and thus 0.1^p for all dimensions.

(d) If p is large, the nearest neighbor to a test input might still be quite far away from the test input: when $p = 1$ in the situation in (a), about 10% of the training data could be expected to be within ± 0.05, and we can thus expect to find an observation 'similar' to the test case among the training data, yielding a hopefully useful prediction. If $p = 10$ in (c), only 0.000001% of the training data could be expected to be within the hypercube ± 0.05 for each dimension around the test input, and it seems less likely that we have an observation among the training data that is 'similar' to the test case, and the prediction performance may therefore deteriorate.

(e)
- For $p = 1$, the side needs to be 0.1.
- For $p = 2$, the side needs to be $0.1^{1/2} = 0.316$.
- For $p = 3$, the side needs to be $0.1^{1/3} = 0.464$.
- ...
- For p, the side needs to be $0.1^{1/p}$.

Thus, if the number of input is high, let us say $p = 100$, the side of the cube needs to be 0.977. This means that if we want on average to use 10% of the training data for a prediction, we would need to include almost the entire range of each input dimension in order to achieve that, because of the large number of dimensions.

Bias-variance trade-off, model selection and cross validation

5.1 **Bias-variance trade-off—understand the concept: regression**

You have access to a European database of 1 000 000 individual trees of various types which include the following entries:

- Tree type (birch, pine, aspen etc.). In total 98 different classes.
- Age
- Height
- Circumference (at 1 meter height)
- Geographical coordinate of the position of the tree
- Vegetation type (openwoodland, mixedwood, highland, wet coniferous etc.)

All parts of Europe are well represented in the data base.

Consider a regression problem where you want to model the age of a tree based on its height and circumference. We use a linear regression model with two input variables

$$y = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \epsilon,$$

where the input variables represent the height and the circumference, and the output is the age.

(a) What causes the bias of the model? Do you think the bias is high or low?

(b) What causes the variance of the model? Do you think the variance is high or low?

(c) What causes the irreducible error of the model?

(d) Where do you see the biggest improvement potential of the model (reducing bias, variance or irreducible error) and how would you go about improving it?

5.2 **Bias-variance trade-off—understand the concept: classification**

Consider the same data base as in Exercise 5.1. Now you consider a classification problem where you model the tree class as the output and the geographical coordinates as the input. We use a k-nearest neighbor model with $k = 1$.

(a) What causes the bias and variance of the model? Do you think the bias and the variance is high or low, respectively?

(b) How could you trade variance for some bias (or vice versa depending on your answer to the previous question) to reduce the mean square error without changing the set of input variables?

Note that we have not presented any formal definition of model bias and model variance for classification problems. However, we can still reason about the concepts in the same manner as we did for the regression setting.

5.3 **Bias and variance when estimating a constant with ridge regression**

To illustrate the concept of bias and variance, let us consider the very simple case of estimating the constant 1 as a linear regression problem. Assume that we have one data sample y_1 ($n = 1$) from

$$y = f_0(\mathbf{x}) + \epsilon, \qquad f_0(\mathbf{x}) = 1,$$

where ϵ has mean 0 and variance σ^2. We use linear regression with only a θ_0-term,

$$y = \theta_0 + \epsilon,$$

where we learn θ_0 using *ridge regression* with regularization parameter λ. Since this problem is so simple that it has no inputs $\mathbf{x}$, the distribution $p(\mathbf{x})$ does not matter.

(a) What is the closed-form solution for $\widehat{\theta}_0$, as a function of the training data y_1 and the regularization parameter λ? What is $\widehat{y}_\star(\mathbf{x}_\star; \mathcal{T})$?

(b) What is the average trained model $\bar{f}(\mathbf{x}_\star) \triangleq \mathbb{E}_\mathcal{T}\left[\widehat{y}_\star(\mathbf{x}_\star; \mathcal{T})\right]$? The expectation operator $\mathbb{E}_\mathcal{T}$ is an expectation over all random variations in the training data.

(c) What is the squared bias $\mathbb{E}_\star\left[(\bar{f}(\mathbf{x}_\star) - f_0(\mathbf{x}_\star))^2\right]$? The expectation operator $\mathbb{E}_\star$ is here an expectation over the test input $\mathbf{x}_\star \sim p(\mathbf{x})$.

(d) What is the variance $\mathbb{E}_\star\left[\mathbb{E}_\mathcal{T}\left[(\widehat{y}(\mathbf{x}_\star; \mathcal{T}) - \bar{f}(\mathbf{x}_\star))^2\right]\right]$?

(e) What is the irreducible error $\mathbb{E}[\epsilon^2]$?

(f) What is $\bar{E}_{\text{new}} = \mathbb{E}_\mathcal{T}\left[\mathbb{E}_\star\left[(\widehat{y}(\mathbf{x}_\star; \mathcal{T}) - y_\star)^2\right]\right]$ for this problem?

(g) For which value of the regularization parameter λ is $\bar{E}_{\text{new}}$ minimized? What does it tell us about the bias-variance trade-off for this (simple) problem?

5.4 Model selection

Suppose that you collect $n = 200$ observations of a single variable x and its single output y. You then go ahead and fit the linear regression model

$$y = \theta_0 + \theta_1 x + \epsilon \tag{5.1}$$

to the first half of your data (the training data), as well as a cubic polynomial

$$y = \theta_0 + \theta_1 x + \theta_2 x^2 + \theta_3 x^3 + \epsilon. \tag{5.2}$$

(a) Suppose that the true relationship between x and y is linear. Which of the models, (5.1) or (5.2), will be able to fit your training data the best? That is, which model gives you the smallest E_{train}?

(b) You now consider the second half of the data (the validation data), which you did not use for training. Using your two previously trained models, which one will be able to predict y in the validation data the best? That is, which model gives you the smallest $E_{\text{hold-out}}$?

(c) Consider (a) and (b) again, but suppose that the true relationship is not linear. Which model will have smallest E_{train} and $E_{\text{hold-out}}$, respectively?

5.5 **Bias and variance when learning a linear function for a quadratic relationship**

As a slightly more involved illustration of the concept of bias and variance, let us consider the case of learning a linear function from data that actually is generated by a quadratic model. Assume that the distribution over data $p(x, y)$ is (implicitly) defined by

$$y = f_0(x), \qquad f_0(x) = x^2, \qquad x \sim \mathcal{U}[-1, 1],$$

from which you randomly observe $n = 2$ independent data points. Those two data point become our training data $\mathcal{T}$. To simplify the calculations, we have restricted ourselves to a problem with no noise ϵ; the only randomness in the problem is for which two input samples x_1 and x_2 we happen to learn about $f_0(x)$ by observing y_1 and y_2.

From the training data with two samples, learn a linear regression model,

$$y = \theta_0 + \theta_1 x + \epsilon,$$

where we assume ϵ is Gaussian.

(a) What is the closed-form solution for $\widehat{y}(x_\star, \mathcal{T})$, as a function of the inputs in the training data x_1, x_2?

(b) What is the average trained model $\bar{f}(x_\star) \triangleq \mathbb{E}_{\mathcal{T}}\left[\widehat{y}_\star(x_\star; \mathcal{T})\right]$?

(c) What is the squared bias $\mathbb{E}_\star\left[(\bar{f}(x_\star) - f_0(x_\star))^2\right]$?

(d) What is the variance $\mathbb{E}_\star\left[\mathbb{E}_{\mathcal{T}}\left[(\widehat{y}(x_\star; \mathcal{T}) - \bar{f}(x_\star))^2\right]\right]$?

(e) What is $\bar{E}_{\text{new}} = \mathbb{E}_{\mathcal{T}}\left[\mathbb{E}_\star\left[(\widehat{y}(x_\star; \mathcal{T}) - y_\star)^2\right]\right]$ for this problem? What would $\bar{E}_{\text{new}}$ be if true relationship had been $f_0(x) = x$ (instead of x^2)?

5.6 For leave-one-out cross validation (or equivalently k-fold cross validation with $k = n$) the cross validation error $E_{\text{k-fold}}$ for ridge regression actually has a closed-form solution

$$E_{\text{k-fold}} = \frac{1}{n}\sum_{i=1}^{n}\left(\frac{\widehat{y}_i - y_i}{1 - [\mathbf{H}(\lambda)]_{ii}}\right)^2, \tag{5.3}$$

where $\widehat{y}_i$ is the prediction of y_i when the model is learned from *all* n data points (no data point is left out), λ the regularization parameter and $[\mathbf{H}(\lambda)]_{ii}$ is element (i, i) of the matrix $\mathbf{H}(\lambda) = \mathbf{X}(\mathbf{X}^\mathsf{T}\mathbf{X} + \lambda I)^{-1}\mathbf{X}^\mathsf{T}$.

In this problem, we will let $\mathbf{x}_i^\mathsf{T}$ denote the entire ith row of $\mathbf{X}$.

(a) Let $\mathbf{X}_{-i}$ denote the matrix $\mathbf{X}$ where row i is removed, and $\mathbf{y}_{-i}$ is the column vector $\mathbf{y}$ with element i removed. Show that

$$\begin{aligned}
\mathbf{X}_{-i}^\mathsf{T}\mathbf{X}_{-i} &= \mathbf{X}^\mathsf{T}\mathbf{X} - \mathbf{x}_i\mathbf{x}_i^\mathsf{T}, \\
\mathbf{X}_{-i}^\mathsf{T}\mathbf{y}_{-i} &= \mathbf{X}^\mathsf{T}\mathbf{y} - \mathbf{x}_i y_i, \text{ and} \\
[\mathbf{H}(\lambda)]_{ii} &= \mathbf{x}_i^\mathsf{T}(\mathbf{X}^\mathsf{T}\mathbf{X} + \lambda I)^{-1}\mathbf{x}_i
\end{aligned}$$

(b) Using the results from (a) and a special case of the matrix inversion lemma

$$(\mathbf{A} - \mathbf{v}\mathbf{v}^\mathsf{T})^{-1} = \mathbf{A}^{-1} + \frac{\mathbf{A}^{-1}\mathbf{v}\mathbf{v}^\mathsf{T}\mathbf{A}^{-1}}{1 - \mathbf{v}^\mathsf{T}\mathbf{A}^{-1}\mathbf{v}},$$

show that

$$\widehat{\boldsymbol{\theta}}_{-i} = \widehat{\boldsymbol{\theta}} + \frac{1}{1 - [\mathbf{H}(\lambda)]_{ii}}(\mathbf{X}^\mathsf{T}\mathbf{X} + \lambda I)^{-1}\mathbf{x}_i(\widehat{y}_i - y_i).$$

Here $\widehat{\boldsymbol{\theta}}_{-i}$ are the parameters learned from all data points except i, and $\widehat{\boldsymbol{\theta}}$ the parameters learned from all data.

Hint: Start from the ridge regression expression for $\widehat{\boldsymbol{\theta}}_{-i}$ as a function of $\mathbf{X}_{-i}$, $\mathbf{y}_{-i}$ and λ.

(c) Use your result from (b) to derive eq. (5.3), starting from

$$E_{\text{k-fold}} = \frac{1}{n}\sum_{i=1}^{n}\left(\widehat{\boldsymbol{\theta}}_{-i}^\mathsf{T}\mathbf{x}_i - y_i\right)^2.$$

(d) Describe (in a few sentences) what eq. (5.3) can be used for in practice.

Solutions

5.1 (a) The bias can be thought of as the error caused by the simplifying assumptions built into the model. In this example, we use a very simple model, which only takes the height and circumference of a tree into account. For a certain class of trees it will most likely do a systematic error in estimating the age, resulting in a possibly high model bias.

(b) The variance is about the stability of the model in response to new training examples. Assume you randomly would split the data set in two halves and estimate the linear regression model for each of the two halves. Each of them would probably get roughly the same estimated parameter $\widehat{\boldsymbol{\theta}}$ and hence also the same predicted output for a new output $f(x_*, \widehat{\boldsymbol{\theta}})$ since the data set is big in comparison to the complexity of the model. Hence, the variance is probably fairly low.

(c) The irreducible error is the error that we cannot reduce even though we would have a very good model trained on an infinite amount of data. This is based on the notion that there are individual age variations amongst the trees that cannot be explained based only on the input variables (features) in the data base. Another source for the irreducible error is the measurement error when measuring the age, height, circumference, etc.

(d) The main problem with the model is probably the high bias. This can be reduced by including more input variables present in the database such as tree class and vegetation type.

5.2 (a) In this model we will classify a new tree according to the class of the closest tree in the training data. This is highly dependent on the selection of the training data. If we would split the data set in two halves and make a k-nearest neighbor model with $k = 1$ for each of these two data sets, it is likely that we would get very different decision boundaries for the two models since we base the predictions on a single training data point. This means that we have a high variance in the model. As for the bias: whether or not this is high or low depends on whether we think that the geographic location alone is sufficiently informative for determining the tree type. If this is the case, then the bias is low, since the 1-NN model can describe very flexible mappings (in this case from "location" to "tree type"). If, however, there is relevant information about the tree type available in the features not used in the model, then this can be viewed as a bias due to under-modeling of the "true" input-output relationship.

(b) A way to reduce the variance would be to increase k such that the classification does not depend on a single data point, but rather on a group of trees in a neighborhood. However, this also increases the bias (since we make a simplifying assumption) and the most common tree class will be favored. For example, in the limit where $k = 1\,000\,000$, all test point would be classified according to the most common class in the data base causing a huge model bias.

5.3 (a) In general, ridge regression is $\widehat{\boldsymbol{\theta}} = \left(\mathbf{X}^\mathsf{T}\mathbf{X} + \lambda\mathbf{I}\right)^{-1}\mathbf{X}^\mathsf{T}\mathbf{y}$, which for our problem gives

$$\widehat{\theta}_0 = \frac{y_1}{1+\lambda}.$$

We have, for this very simple problem, that $\widehat{y}_\star(\mathbf{x}_\star; \mathcal{T}) = \widehat{\theta}_0$.

(b) We have

$$\bar{f}(\mathbf{x}_\star; \mathcal{T}) = \mathbb{E}_\mathcal{T}\left[\widehat{y}_\star(\mathbf{x}_\star; \mathcal{T})\right] = \mathbb{E}_\mathcal{T}\left[\frac{y_1}{1+\lambda}\right] = \mathbb{E}_\mathcal{T}\left[\frac{1+\epsilon}{1+\lambda}\right] = \frac{1}{1+\lambda}(1 + \underbrace{\mathbb{E}_\mathcal{T}\left[\epsilon\right]}_{0}) = \frac{1}{1+\lambda}$$

(c) The squared bias is

$$\mathbb{E}_\star\left[(\bar{f}(\mathbf{x}_\star) - f_0(\mathbf{x}_\star))^2\right] = \mathbb{E}_\star\left[\left(\frac{1}{1+\lambda} - 1\right)^2\right] = \left(\frac{1}{1+\lambda} - 1\right)^2 = \left(\frac{\lambda}{1+\lambda}\right)^2$$

(d) The variance is

$$\begin{aligned}\mathbb{E}_\star\left[\mathbb{E}_\mathcal{T}\left[(\widehat{y}(\mathbf{x}_\star; \mathcal{T}) - \bar{f}(\mathbf{x}_\star))^2\right]\right] &= \mathbb{E}_\star\left[\mathbb{E}_\mathcal{T}\left[\left(\frac{1+\epsilon}{1+\lambda} - \frac{1}{1+\lambda}\right)^2\right]\right] = \mathbb{E}_\star\left[\mathbb{E}_\mathcal{T}\left[\left(\frac{\epsilon}{1+\lambda}\right)^2\right]\right] \\ &= \frac{1}{(1+\lambda)^2}\mathbb{E}_\star\left[\mathbb{E}_\mathcal{T}\left[\epsilon^2\right]\right] = \frac{\sigma^2}{(1+\lambda)^2}\end{aligned}$$

(e) The irreducible error is $\mathbb{E}[\epsilon^2] = \sigma^2$.

(f) From the lecture notes, we have that $\bar{E}_{\text{new}}$ = squared bias + variance + irreducible error, hence

$$\bar{E}_{\text{new}} = \left(\frac{\lambda}{1+\lambda}\right)^2 + \frac{\sigma^2}{(1+\lambda)^2} + \sigma^2.$$

(g) By differentiating $\bar{E}_{\text{new}}$ with respect to λ, we get

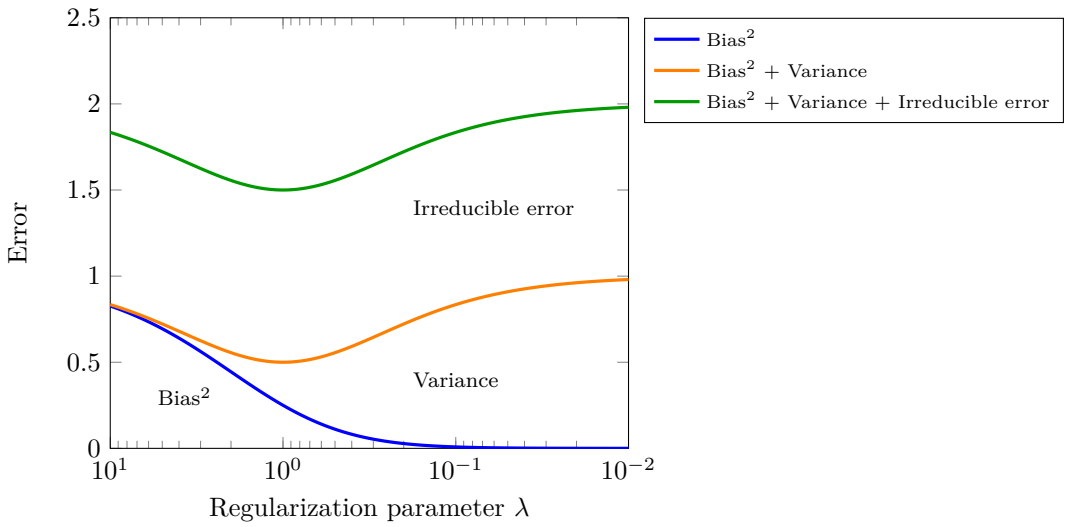
$$\frac{\partial}{\partial\lambda}\bar{E}_{\text{new}} = \frac{2(\lambda-\sigma^2)}{(1+\lambda)^3}.$$

We conclude that for this particular problem, $\bar{E}_{\text{new}}$ is minimized when $\lambda = \sigma^2$. In terms of bias-variance trade-off, we can see that for this problem

i. There is no bias when $\lambda = 0$, but the variance peaks at σ^2

ii. There is no variance when $\lambda \to \infty$, but the bias peaks at 1

iii. The smallest expected new data error, $\bar{E}_{\text{new}}$, is achieved at neither of those extremes, but at $\lambda = \sigma^2$. For $\sigma^2 = 1$, we can make the following plot:



5.4 (a) Since (5.2) has more flexibility than (5.1) (note that if you set $\theta_2 = \theta_3 = 0$ in (5.2), you get (5.1)), it will be able to fit to the training data *at least as good* as (5.1). Thus, E_{train} for (5.2) is $\leq$ than E_{train} for (5.1).

(b) (5.2) will most likely overfit to the training data, since the model is more flexible than the true relationship between the input and output. Thus, $E_{\text{hold-out}}$ for (5.2) is likely to be $\geq$ than $E_{\text{hold-out}}$ for (5.1).

(c) The argument for (a) is still applicable in the training case, i.e., E_{train} for (5.2) is $\leq$ than E_{train} for (5.1). For $E_{\text{hold-out}}$, we cannot tell unless we have more information about the true relationship between the input and the output.

5.5 (a) By denoting $\mathbf{y} = [x_1^2\ x_2^2]^\mathsf{T}$, one way to find the expression is to analytically solve $\mathbf{X}\widehat{\boldsymbol{\theta}} = \mathbf{y}$,

$$\begin{bmatrix} 1 & x_1 \\ 1 & x_2 \end{bmatrix}\begin{bmatrix}\widehat{\theta}_0 \\ \widehat{\theta}_1\end{bmatrix} = \begin{bmatrix} x_1^2 \\ x_2^2\end{bmatrix} \Rightarrow \cdots \Rightarrow \begin{bmatrix}\widehat{\theta}_0 \\ \widehat{\theta}_1\end{bmatrix} = \begin{bmatrix} -x_1x_2 \\ x_1+x_2\end{bmatrix}.$$

This gives

$$\widehat{y}(x_\star;\mathcal{T}) = \widehat{\theta}_0 + \widehat{\theta}_1 x_\star = -x_1x_2 + (x_1+x_2)x_\star.$$

(b) Since the only random element of the training data is its two input samples x_1, x_2, which both have a uniform distribution on $[-1, 1]$, the average trained model $\bar{f}(x)$ is

$$\bar{f}(x_\star) = \mathbb{E}_{\mathcal{T}}\left[\widehat{y}(x_\star; \mathcal{T})\right] = \iint \widehat{y}(x_\star; x_1, x_2) \underbrace{p(x_1, x_2)}_{\frac{1}{4} \text{ on } [-1,1]^2} dx_1 dx_2 = \frac{1}{2} \cdot \frac{1}{2} \int_{-1}^{1} \int_{-1}^{1} \widehat{y}(x_\star; x_1, x_2) dx_1 dx_2 =$$

$$= \frac{1}{4} \int_{-1}^{1} \int_{-1}^{1} -x_1 x_2 + (x_1 + x_2) x_\star dx_1 dx_2 = 0$$

(c) The squared bias is

$$\mathbb{E}_\star\left[(\bar{f}(x_\star) - f_0(x_\star))^2\right] = \mathbb{E}_\star\left[(0 - x_\star^2)^2\right] = \frac{1}{2} \int_{-1}^{1} x_\star^4 dx_\star = \frac{1}{5}.$$

(d) The variance is

$$\begin{aligned}
\mathbb{E}_\star\left[\mathbb{E}_{\mathcal{T}}\left[(\widehat{y}(x_\star; \mathcal{T}) - \bar{f}(x_\star))^2\right]\right] &= \mathbb{E}_\star\left[\mathbb{E}_{\mathcal{T}}\left[(x_1 x_2 - (x_1 + x_2) x_\star)^2\right]\right] \\
&= \mathbb{E}_\star\left[\mathbb{E}_{\mathcal{T}}\left[x_1^2 x_2^2\right] - 2 x_\star \mathbb{E}_{\mathcal{T}}\left[x_1^2 x_2^2 (x_1 + x_2)\right] + x_\star^2 \mathbb{E}_{\mathcal{T}}\left[(x_1 + x_2)^2\right]\right] \\
&= \mathbb{E}_{\mathcal{T}}\left[x_1^2 x_2^2\right] - 2 \underbrace{\mathbb{E}_\star\left[x_\star\right]}_{0} \mathbb{E}_{\mathcal{T}}\left[x_1^2 x_2^2 (x_1 + x_2)\right] + \mathbb{E}_\star\left[x_\star^2\right] \mathbb{E}_{\mathcal{T}}\left[(x_1 + x_2)^2\right] \\
&= \frac{1}{4} \int_{-1}^{1} \int_{-1}^{1} x_1^2 x_2^2 dx_1 dx_2 + \left(\frac{1}{2} \int_{-1}^{1} x_\star^2 dx_\star\right) \left(\frac{1}{4} \int_{-1}^{1} \int_{-1}^{1} (x_1 + x_2)^2 dx_1 dx_2\right) \\
&= \frac{1}{9} + \frac{1}{3} \cdot \frac{2}{3} = \frac{1}{3}.
\end{aligned}$$

(e) Since there is no irreducible error, we have that $\bar{E}_{\text{new}}$ = squared bias + variance (this can be thoroughly derived using the definition of $\bar{E}_{\text{new}}$),

$$\bar{E}_{\text{new}} = \frac{1}{5} + \frac{1}{3} = \frac{8}{15}.$$

This is not due to any noise (there is no noise in this problem!), but only the fact that the model $(\theta_0 + \theta_1 x)$ can not describe the 'reality' $(f_0(x) = x^2)$ perfectly well. If the 'reality' had been $f_0(x) = x$, there would have been no bias or variance (because there is no noise), and hence $\bar{E}_{\text{new}} = 0$.

5.6 (a) We have

$$\mathbf{X}_{-i}^\mathsf{T} \mathbf{X}_{-i} = \sum_{j=1, j\neq i}^{n} \mathbf{x}_j \mathbf{x}_j^\mathsf{T} = \left(\sum_{j=1}^{n} \mathbf{x}_j \mathbf{x}_j^\mathsf{T}\right) - \mathbf{x}_i \mathbf{x}_i^\mathsf{T} = \mathbf{X}^\mathsf{T}\mathbf{X} - \mathbf{x}_i \mathbf{x}_i^\mathsf{T}$$

and similarly

$$\mathbf{X}_{-i}^\mathsf{T} \mathbf{y}_{-i} = \sum_{j=1, j\neq i}^{n} \mathbf{x}_j y_j = \left(\sum_{j=1}^{n} \mathbf{x}_j y_j^\mathsf{T}\right) - \mathbf{x}_i y_i = \mathbf{X}^\mathsf{T}\mathbf{y} - \mathbf{x}_i y_i.$$

For matrices $\mathbf{A} \in \mathbb{R}^{n \times n}$ and $i \in \{1, \ldots, n\}$ we can express element $[\mathbf{A}]_{ii}$ as $[\mathbf{A}]_{ii} = \mathbf{e}_i^\mathsf{T} \mathbf{A} \mathbf{e}_i$, where $\mathbf{e}_i \in \mathbb{R}^n$ is the ith unit vector (its ith element is 1 and all other entries are 0). Thus we get

$$\begin{aligned}
[\mathbf{H}(\lambda)]_{ii} = \mathbf{e}_i^\mathsf{T} \mathbf{H}(\lambda) \mathbf{e}_i &= \mathbf{e}_i^\mathsf{T} \left(\mathbf{X}(\mathbf{X}^\mathsf{T}\mathbf{X} + \lambda I)^{-1} \mathbf{X}^\mathsf{T}\right) \mathbf{e}_i \\
&= \left(\mathbf{e}_i^\mathsf{T} \mathbf{X}\right) (\mathbf{X}^\mathsf{T}\mathbf{X} + \lambda I)^{-1} \left(\mathbf{X}^\mathsf{T} \mathbf{e}_i\right) = \mathbf{x}_i^\mathsf{T} (\mathbf{X}^\mathsf{T}\mathbf{X} + \lambda I)^{-1} \mathbf{x}_i,
\end{aligned}$$

where we used that $\mathbf{X}^\mathsf{T} \mathbf{e}_i = \mathbf{x}_i$.

(b) From the ridge regression expression we know that the parameters $\widehat{\boldsymbol{\theta}}_{-i}$ learned from all data points except i are given by

$$\widehat{\boldsymbol{\theta}}_{-i} = \left(\mathbf{X}_{-i}^\mathsf{T} \mathbf{X}_{-i} + \lambda I\right)^{-1} \mathbf{X}_{-i}^\mathsf{T} \mathbf{y}_{-i}.$$

Using the first two equations from (a) we get

$$\widehat{\boldsymbol{\theta}}_{-i} = \left(\mathbf{X}^\mathsf{T}\mathbf{X} - \mathbf{x}_i \mathbf{x}_i^\mathsf{T} + \lambda I\right)^{-1} (\mathbf{X}^\mathsf{T}\mathbf{y} - \mathbf{x}_i y_i). \tag{5.4}$$

Applying the matrix inversion lemma with $\mathbf{A} = \mathbf{X}^\mathsf{T}\mathbf{X} + \lambda I$ and $\mathbf{v} = \mathbf{x}_i$ and using the third equation from (a) yields

$$\begin{aligned}\left(\mathbf{X}^\mathsf{T}\mathbf{X} - \mathbf{x}_i\mathbf{x}_i^\mathsf{T} + \lambda I\right)^{-1} &= \left(\mathbf{X}^\mathsf{T}\mathbf{X} + \lambda I\right)^{-1} + \frac{\left(\mathbf{X}^\mathsf{T}\mathbf{X} + \lambda I\right)^{-1}\mathbf{x}_i\mathbf{x}_i^\mathsf{T}\left(\mathbf{X}^\mathsf{T}\mathbf{X} + \lambda I\right)^{-1}}{1 - \mathbf{x}_i^\mathsf{T}\left(\mathbf{X}^\mathsf{T}\mathbf{X} + \lambda I\right)^{-1}\mathbf{x}_i} \\ &= \left(\mathbf{X}^\mathsf{T}\mathbf{X} + \lambda I\right)^{-1} + \frac{\left(\mathbf{X}^\mathsf{T}\mathbf{X} + \lambda I\right)^{-1}\mathbf{x}_i\mathbf{x}_i^\mathsf{T}\left(\mathbf{X}^\mathsf{T}\mathbf{X} + \lambda I\right)^{-1}}{1 - [\mathbf{H}(\lambda)]_{ii}}.\end{aligned}$$

If we plug this result in eq. (5.4) and use the third equation from (a) we get

$$\begin{aligned}\widehat{\boldsymbol{\theta}}_{-i} &= \left(\left(\mathbf{X}^\mathsf{T}\mathbf{X} + \lambda I\right)^{-1} + \frac{\left(\mathbf{X}^\mathsf{T}\mathbf{X} + \lambda I\right)^{-1}\mathbf{x}_i\mathbf{x}_i^\mathsf{T}\left(\mathbf{X}^\mathsf{T}\mathbf{X} + \lambda I\right)^{-1}}{1 - [\mathbf{H}(\lambda)]_{ii}}\right)(\mathbf{X}^\mathsf{T}\mathbf{y} - \mathbf{x}_i y_i) \\ &= \left(\mathbf{X}^\mathsf{T}\mathbf{X} + \lambda I\right)^{-1}\mathbf{X}^\mathsf{T}\mathbf{y} + \frac{1}{1 - [\mathbf{H}(\lambda)]_{ii}}\left(\mathbf{X}^\mathsf{T}\mathbf{X} + \lambda I\right)^{-1}\mathbf{x}_i\mathbf{x}_i^\mathsf{T}\left(\mathbf{X}^\mathsf{T}\mathbf{X} + \lambda I\right)^{-1}\mathbf{X}^\mathsf{T}\mathbf{y} \\ &\quad - \left(\mathbf{X}^\mathsf{T}\mathbf{X} + \lambda I\right)^{-1}\mathbf{x}_i\left(1 + \frac{\mathbf{x}_i^\mathsf{T}\left(\mathbf{X}^\mathsf{T}\mathbf{X} + \lambda I\right)^{-1}\mathbf{x}_i}{1 - [\mathbf{H}(\lambda)]_{ii}}\right)y_i \\ &= \left(\mathbf{X}^\mathsf{T}\mathbf{X} + \lambda I\right)^{-1}\mathbf{X}^\mathsf{T}\mathbf{y} + \frac{1}{1 - [\mathbf{H}(\lambda)]_{ii}}\left(\mathbf{X}^\mathsf{T}\mathbf{X} + \lambda I\right)^{-1}\mathbf{x}_i\mathbf{x}_i^\mathsf{T}\left(\mathbf{X}^\mathsf{T}\mathbf{X} + \lambda I\right)^{-1}\mathbf{X}^\mathsf{T}\mathbf{y} \\ &\quad - \frac{1}{1 - [\mathbf{H}(\lambda)]_{ii}}\left(\mathbf{X}^\mathsf{T}\mathbf{X} + \lambda I\right)^{-1}\mathbf{x}_i y_i \\ &= \left(\mathbf{X}^\mathsf{T}\mathbf{X} + \lambda I\right)^{-1}\mathbf{X}^\mathsf{T}\mathbf{y} \\ &\quad + \frac{1}{1 - [\mathbf{H}(\lambda)]_{ii}}\left(\mathbf{X}^\mathsf{T}\mathbf{X} + \lambda I\right)^{-1}\mathbf{x}_i\left(\mathbf{x}_i^\mathsf{T}\left(\mathbf{X}^\mathsf{T}\mathbf{X} + \lambda I\right)^{-1}\mathbf{X}^\mathsf{T}\mathbf{y} - y_i\right).\end{aligned}$$

The parameters $\widehat{\boldsymbol{\theta}}$ learned from all data points satisfy

$$\widehat{\boldsymbol{\theta}} = \left(\mathbf{X}^\mathsf{T}\mathbf{X} + \lambda I\right)^{-1}\mathbf{X}^\mathsf{T}\mathbf{y},$$

and hence the model learned from all data points predicts the value

$$\widehat{y}_i = \widehat{\boldsymbol{\theta}}^\mathsf{T}\mathbf{x}_i = \mathbf{x}_i^\mathsf{T}\widehat{\boldsymbol{\theta}} = \mathbf{x}_i^\mathsf{T}\left(\mathbf{X}^\mathsf{T}\mathbf{X} + \lambda I\right)^{-1}\mathbf{X}^\mathsf{T}\mathbf{y}$$

for the ith data point. Thus we obtain

$$\widehat{\boldsymbol{\theta}}_{-i} = \widehat{\boldsymbol{\theta}} + \frac{1}{1 - [\mathbf{H}(\lambda)]_{ii}}\left(\mathbf{X}^\mathsf{T}\mathbf{X} + \lambda I\right)^{-1}\mathbf{x}_i\left(\widehat{y}_i - y_i\right).$$

(c) Using the result from (b) we get

$$\begin{aligned}E_{\text{k-fold}} &= \frac{1}{n}\sum_{i=1}^{n}\left(\widehat{\boldsymbol{\theta}}_{-i}^\mathsf{T}\mathbf{x}_i - y_i\right)^2 \\ &= \frac{1}{n}\sum_{i=1}^{n}\left(\left(\widehat{\boldsymbol{\theta}} + \frac{1}{1 - [\mathbf{H}(\lambda)]_{ii}}\left(\mathbf{X}^\mathsf{T}\mathbf{X} + \lambda I\right)^{-1}\mathbf{x}_i\left(\widehat{y}_i - y_i\right)\right)^\mathsf{T}\mathbf{x}_i - y_i\right)^2 \\ &= \frac{1}{n}\sum_{i=1}^{n}\left(\left(\widehat{\boldsymbol{\theta}}^\mathsf{T} + \frac{1}{1 - [\mathbf{H}(\lambda)]_{ii}}\mathbf{x}_i^\mathsf{T}\left(\mathbf{X}^\mathsf{T}\mathbf{X} + \lambda I\right)^{-1}\left(\widehat{y}_i - y_i\right)\right)\mathbf{x}_i - y_i\right)^2 \\ &= \frac{1}{n}\sum_{i=1}^{n}\left(\widehat{\boldsymbol{\theta}}^\mathsf{T}\mathbf{x}_i + \frac{1}{1 - [\mathbf{H}(\lambda)]_{ii}}\mathbf{x}_i^\mathsf{T}\left(\mathbf{X}^\mathsf{T}\mathbf{X} + \lambda I\right)^{-1}\mathbf{x}_i\left(\widehat{y}_i - y_i\right) - y_i\right)^2.\end{aligned}$$

Since $\widehat{y}_i = \widehat{\boldsymbol{\theta}}^{\mathsf{T}} \mathbf{x}_i$ and from (a) we know that $[\mathbf{H}(\lambda)]_{ii} = \mathbf{x}_i^{\mathsf{T}} \left(\mathbf{X}^{\mathsf{T}} \mathbf{X} + \lambda I \right)^{-1} \mathbf{x}_i$, we obtain

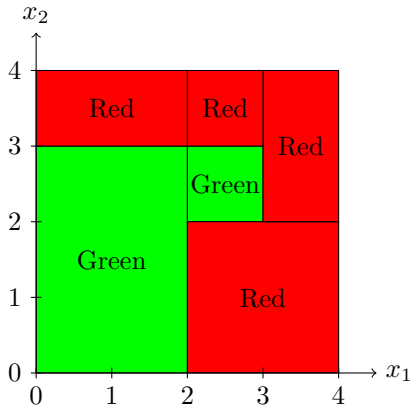
$$\begin{aligned} E_{\text{k-fold}} &= \frac{1}{n} \sum_{i=1}^{n} \left(\hat{y}_i + \frac{1}{1 - [\mathbf{H}(\lambda)]_{ii}} [\mathbf{H}(\lambda)]_{ii} \left(\widehat{y}_i - y_i \right) - y_i \right)^2 \\ &= \frac{1}{n} \sum_{i=1}^{n} \left(\frac{\widehat{y}_i - y_i}{1 - [\mathbf{H}(\lambda)]_{ii}} \right)^2 . \end{aligned}$$

(d) Ideally, we would like to choose regularization parameter λ in ridge regression such that the new data error of the resulting trained model is minimized. Unfortunately, we can not usually compute the new data error. The cross validation error $E_{\text{k-fold}}$ provides an approximation of the new data error, and hence can be used to select a "good" λ. However, performing k-fold cross validation can be computationally expensive and time consuming, since usually it requires training the model from scratch and evaluating its performance c times. In particular, performing leave-one-out cross validation with $c = n$ might not be feasible. In ridge regression, however, eq. (1) allows us to train the model only once on the full dataset and to compute the leave-one-out cross validation error from its prediction errors without having to retrain the model n times on all subsets of our dataset with $n - 1$ samples.

Tree-Based Methods

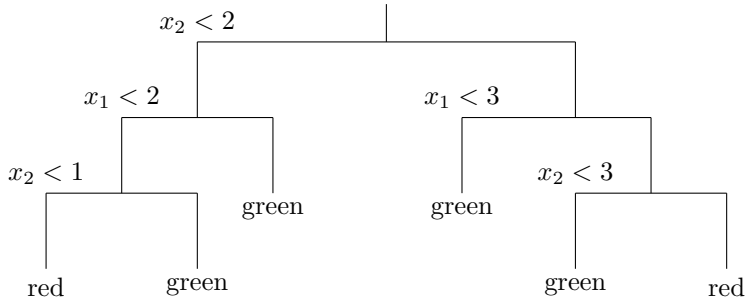
7.1 Basics of decision tree

(a) Sketch (by hand) the classification tree corresponding to the following partition:



(b) How many terminal nodes (leaves) and internal nodes does the resulting tree have?

(c) Sketch (by hand) the partition of $[0, 4] \times [0, 4]$ for the following classification tree



7.2 Bootstrapping for bagging

Bagging starts with boostrapping B datasets $\{\widetilde{\mathcal{T}}^b\}_{b=1}^B$ from the original training dataset $\mathcal{T}$ (i.e., independently drawing data points with replacement from $\mathcal{T}$). Let us say that all data the original dataset $\mathcal{T}$ as well as the boostrapped ones $\mathcal{T}^b$ contains n data points.

(a) What is the probability that the first data point in the first bootstrapped dataset $\widetilde{\mathcal{T}}^1$ is *not* the ith data point from the original dataset $\mathcal{T}$?

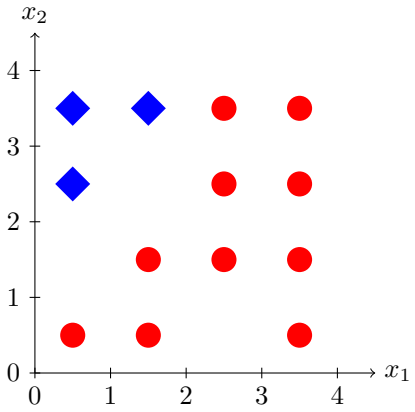
(b) What is the probability that the first dataset $\widetilde{\mathcal{T}}^1$ does *not* contain the ith data point from the original dataset $\mathcal{T}$ at all?

(c) In the limit as $n \to \infty$, how many of the data points in $\mathcal{T}$ can you expect to find replicated at least once in $\widetilde{\mathcal{T}}^1$?
Hint: $(1 + x/n)^n \to e^x$ *as* $n \to \infty$

(d) As we have seen above a certain data point in $\mathcal{T}$ is not present in all bootstrapped datasets $\widetilde{\mathcal{T}}^1, \ldots, \widetilde{\mathcal{T}}^B$. How can this be used instead of cross-validation in bagging methods?

7.3 Tree construction: classification error rate vs. Gini index

Consider the following data (different colors and shape represent different classes), for which we are going to construct a classification tree:



There are several alternatives in how to decide which splits to make when constructing a classification tree. Trees are often constructed by adding one split at a time to the tree, and terminate the construction when some criterion is reached. When deciding which split to add to the tree, a cost function

$$\sum_{\ell=1}^{|T|} n_\ell Q_\ell \tag{7.1}$$

is considered, where T is the tree object, $|T|$ is the total number of terminal nodes in the tree, n_ℓ is the number of training data points in the region corresponding to the terminal node ℓ, and Q_ℓ is a node impurity measure of the same region. Two common measures are

$$\text{Classification error rate: } Q_\ell = 1 - \max_m \hat{\pi}_{\ell m}, \tag{7.2}$$

$$\text{Gini index: } Q_\ell = \sum_{m=1}^{M} \hat{\pi}_{\ell m}(1 - \hat{\pi}_{\ell m}), \tag{7.3}$$

where $\hat{\pi}_{\ell m}$ is the proportion of training data points with label m in the region corresponding to node ℓ. Since the classification for any data point within a certain region is decided with a 'majority vote', the proportion of correct training data classifications in region ℓ can thus be expressed as $\max_m \hat{\pi}_{\ell m}$, and the proportion of misclassifications as $1 - \max_\ell \hat{\pi}_{\ell m}$.

(a) Consider the problem of making the first split when building a tree. Where would you make the split?

(i) Find the single split which minimizes the cost function (7.1) when considering the classification error rate.

(ii) Find the single split which minimizes the cost function (7.1) when considering the Gini index.

Which split would be the optimal (counting the number of misclassifications) if you would terminate the tree-construction now, and be satisfied with this depth-1 tree (i.e., stump)?

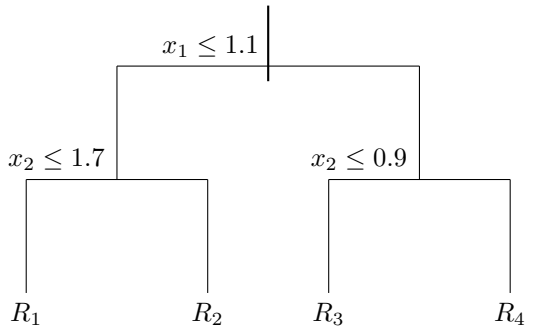
(b) Now add another split to your trees (i) and (ii), respectively. Which tree seems to be the preferable? Explain your findings!

7.4 **Regression trees**

Consider a regression problem with a two input variables $\mathbf{x} = [x_1\ x_2]^\mathsf{T}$, where $x_1 \in [0, 3]$ and $x_2 \in [0, 3]$, and one output $y \in \mathbb{R}$. Based on the following training data

i	x_1	x_2	y
1	1.4	1.4	0.1
2	2.2	2.2	0.6
3	0.2	0.8	-0.7
4	1.0	2.8	-1.8
5	0.6	0.2	0.3
6	0.4	2.2	-1.9
7	0.6	1.4	-0.9
8	1.2	1.8	-0.5
9	1.2	0.6	0.7
10	1.8	0.6	1.5

the regression tree shown below has been constructed using recursive binary splitting.



(a) Draw the corresponding input partitioning to this tree. Mark the regions with the names of the leaf nodes $R_1, \ldots, R_4$.

(b) Use the regression tree to predict the output of the test input $\mathbf{x}_\star = [1.5\ 1.8]^\mathsf{T}$

(c) Continue to grow the tree by adding splits such that there are at most two data points in each region. Choose your splits in order to minimize the residual sum of squares (RSS)

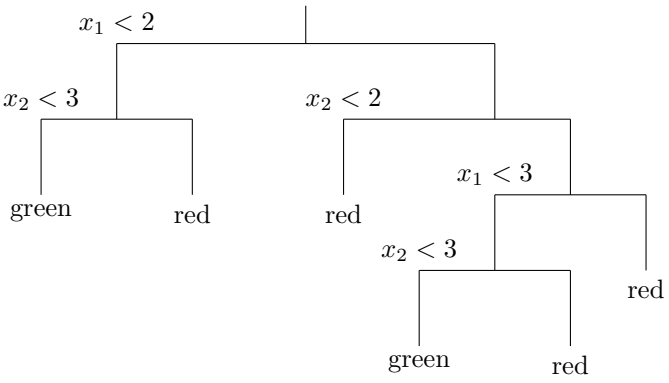
$$\sum_{j=1}^{J} \sum_{i \in R_j} (y_i - \widehat{y}_{R_j})^2, \tag{7.4}$$

where J is the number of regions/terminal nodes and $\widehat{y}_{R_j}$ is the mean value of all training outputs in that region (which also is the prediction for test points in that region). Which splits do you make?

(d) What is the predicted output of the test input $x_\star$ from (b) using this new, deeper tree?

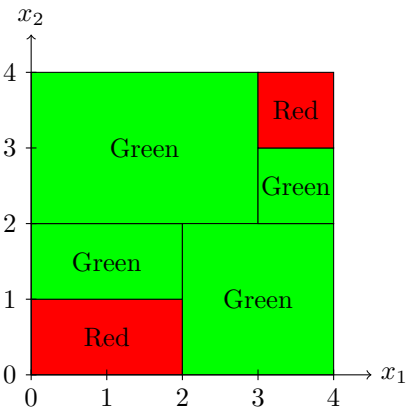
Solutions

7.1 (a)



(b) The tree has six terminal nodes and five internal nodes.

(c)



7.2 (a) The probability that the first data point in $\widetilde{\mathcal{T}}^1$ is the ith data point from $\mathcal{T}$ is $1/n$, hence is its complement $1-1/n$.

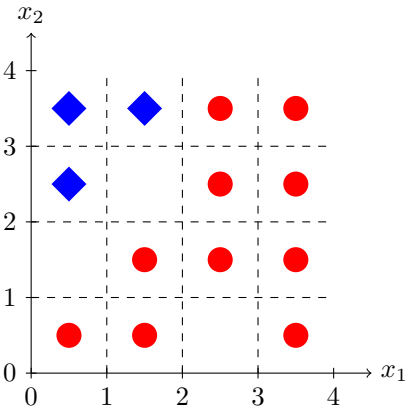
(b) Since the data points in $\widetilde{\mathcal{T}}^1$ are drawn independently from $\mathcal{T}$, the probability for the ith data point never to be drawn in n draws is $(1-1/n)^n$.

(c) Since $(1-1/n)^n \to e^{-1} = 1/e$ as $n \to \infty$, the probability for the ith data point from $\mathcal{T}$ not to be replicated in $\widetilde{\mathcal{T}}^1$ is $1/e$, and we hence expect that $1-1/e \approx 63\%$ of the data points from $\mathcal{T}$ to be replicated in $\widetilde{\mathcal{T}}^1$ on the average.

(d) Remember, the main idea in validation is to see how well a method performs on previously unseen data points (validation data or test data). In cross validation, this is achieved by holding out a fraction of the training data, and use it as validation data instead. In bagging, a similar effect can be achieved without the extra computational burden of cross validation (useful in situations when the computational power is scarce).

In bagging, a separate model is learned for each bagged dataset $\widetilde{\mathcal{T}}^b$, and the final prediction is made as an average over the predictions from *all* B models. If we want to estimate the quality of the prediction for the ith data point in the original set $\mathcal{T}$ without using cross-validation, we can make a prediction by averaging only over the approximately $(1-0.63)B = 0.37B$ models estimated without the ith data point, and compare it to the true y_i.

7.3 (a) There are infinitely many possible splits we could make, but if we restrict ourselves to splits at integer values in the region where the training data is located, we have 6 sensible options (either at $x_1 = 1, 2$ or 3 or $x_2 = 1, 2$ or 3).



To avoid unnecessary calculations, we can start by ruling out $x_1 = 3$ and $x_2 = 1$, since they are clearly suboptimal.

(i) When considering misclassification rate $Q_\ell(T) = 1 - \max_m \widehat{\pi}_{\ell m}$, the cost function take the following values for the different splits:

Split	n_1	$\widehat{\pi}_{1B}$	$\widehat{\pi}_{1R}$	$Q_1(T)$	n_2	$\widehat{\pi}_{2B}$	$\widehat{\pi}_{2R}$	$Q_2(T)$	$n_1Q_1(T) + n_2Q_2(T)$
$x_1 < 1$	3	2/3	1/3	1/3	10	1/10	9/10	1/10	2
$x_1 < 2$	6	3/6	3/6	3/6	7	0/7	7/7	0	3
$x_2 < 2$	6	0/6	6/6	0	7	3/7	4/7	3/7	3
$x_2 < 3$	9	1/9	8/9	1/9	4	2/4	2/4	2/4	3

Thus, we take the decision to make the split at $x_1 < 1$.

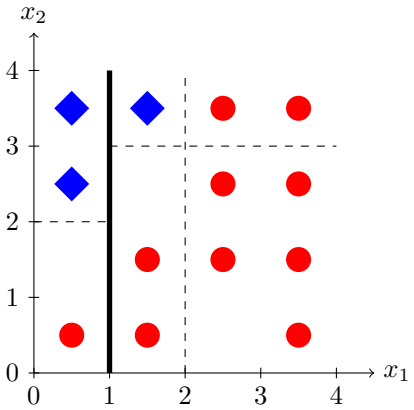
(ii) When considering the Gini index $Q_\ell(T) = \sum_{m=1}^{M} \widehat{\pi}_{\ell m}(1 - \widehat{\pi}_{\ell m})$, the cost function take the following values for the different splits:

Split	n_1	$\widehat{\pi}_{1B}$	$\widehat{\pi}_{1R}$	$Q_1(T)$	n_2	$\widehat{\pi}_{2B}$	$\widehat{\pi}_{2R}$	$Q_2(T)$	$n_1Q_1(T) + n_2Q_2(T)$
$x_1 < 1$	3	2/3	1/3	4/9	10	1/10	9/10	9/50	3.13
$x_1 < 2$	6	3/6	3/6	1/2	7	0/7	7/7	0	3
$x_2 < 2$	6	0/6	6/6	0	7	3/7	4/7	24/49	3.43
$x_2 < 3$	9	1/9	8/9	16/81	4	2/4	2/4	1/2	3.78

Thus, we take the decision to make the split at $x_1 = 2$.

If this is the only decision we are going to make, the decision based on the classification error rate is also the optimal decision for minimizing the classification error rate (only two training data points are misclassified).

(b) (i) For the misclassification-index based tree, the relevant splits to make are the ones marked with dashed lines (again, other splits are also possible, but will clearly be suboptimal):

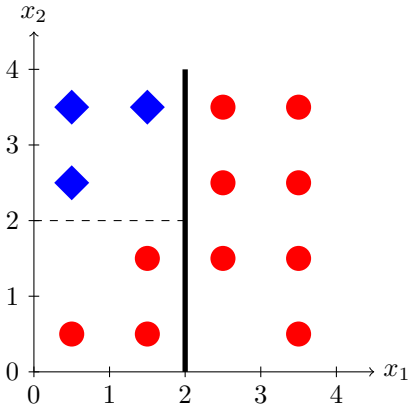


The corresponding cost function will take the following values for the different splits:

Split	n_1	$\widehat{\pi}_{1B}$	$\widehat{\pi}_{1R}$	$Q_1(T)$	n_2	$\widehat{\pi}_{2B}$	$\widehat{\pi}_{2R}$	$Q_2(T)$	n_3	$\widehat{\pi}_{3B}$	$\widehat{\pi}_{3R}$	$Q_3(T)$	$n_1Q_1(T)+n_2Q_2(T)+n_3Q_3(T)$
$x_2 < 2$ ($x_1 \leq 1$)	2	2/2	0/2	0	1	0/1	1/1	0	10	1/10	9/10	1/10	1
$x_2 < 3$ ($x_1 > 1$)	3	1/3	2/3	1/3	3	1/3	2/3	1/3	7	0/7	7/7	0	2
$x_1 < 2$ ($x_1 > 1$)	3	1/3	2/3	1/3	3	1/3	2/3	1/3	7	0/7	7/7	0	2

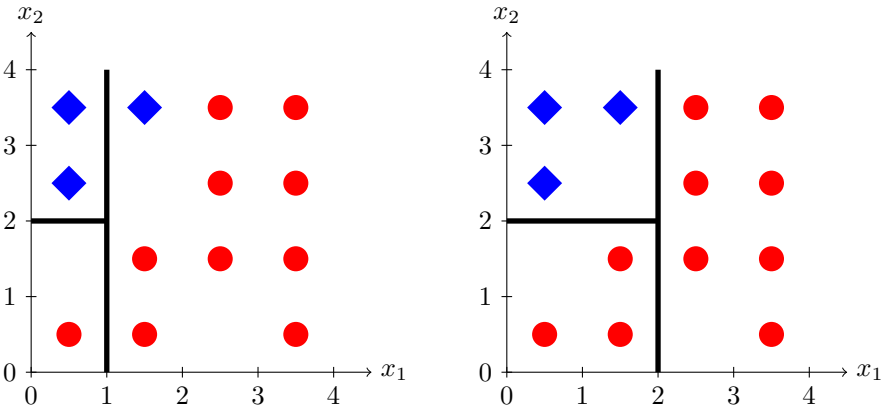
Thus, the optimal decision (with respect to the misclassification rate) is to make the split at $x < 2 = 2$ in the $x_1 < 1$-region.

(ii) For the Gini-index based tree, the only sensible split to make is:



The cost function with the Gini-index is 0 with a split at $x_2 = 2$, and it is thus optimal.

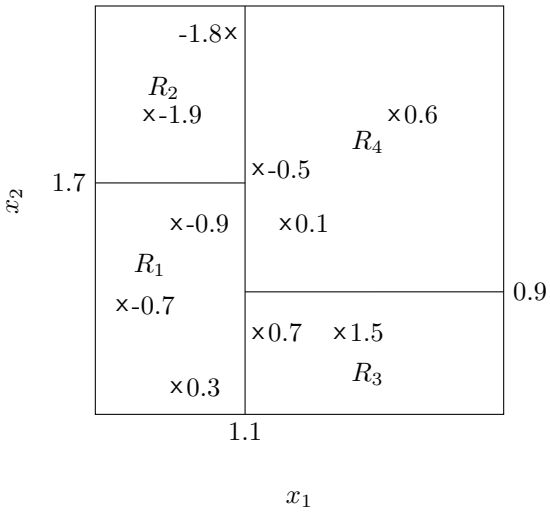
The trees obtained after two splittings are thus



Clearly, the right tree (obtained by the Gini-index based cost function) is preferred over the left (obtained by the cost function based on the classification error rate). The reason is that the sequential splitting algorithm is 'greedy' or 'myopic', by making sequentially decisions only looking one step ahead. The Gini index, however, counteracts

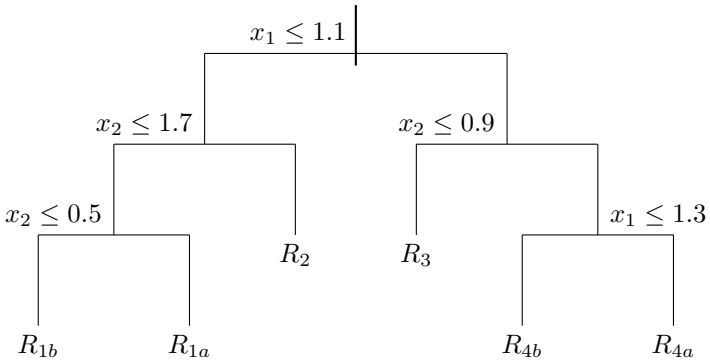
this by promoting splits that gives one relatively pure region (few misclassifications) and another more mixed region (more misclassifications), which later can be split again.

7.4 (a) The partitioning of the input space is as follows

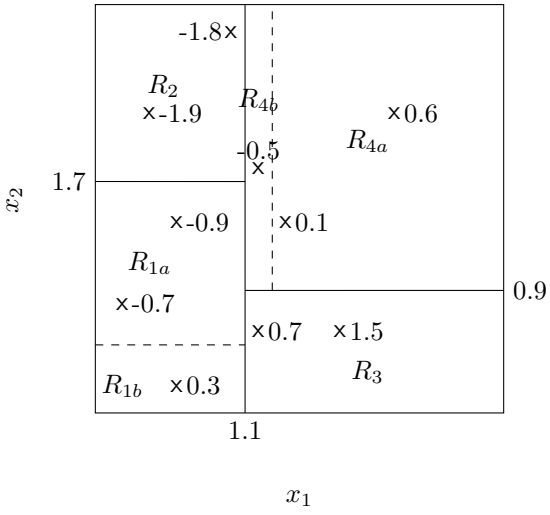


(b) Since $x_{\star 1} = 1.5 > 1.1$ and $x_{\star 2} = 1.8 > 0.9$, the test point belongs to region R_4. To compute the predicted output we also need to know which regions the training data points fall into. This is also shown in the figure above, from which it is clear that data points $i = 1, 2, 8$ are in R_4. The mean of these points is $\widehat{y}_{R_4} = 0.07$, which thus becomes our prediction $\widehat{y}_\star$.

(c) Regions R_2 and R_3 already have two or fewer data points, whereas region R_1 and R_4 need another split, respectively. We find that the RSS-minimizing split of R_1 is for somewhere between $x_2 = 0.2$ and $x_2 = 0.8$, and for R_4 somewhere between $x_1 = 1.2$ and $x_1 = 1.4$, as for example



This deeper tree thus corresponds to a partitioning which looks like



(d) $\mathbf{x}_\star$ now belongs to R_{4a}, which gives $\widehat{y}_\star = \widehat{y}_{R_{4a}} = 0.35$.

Neural networks

10.1 **Draw a neural network**

Draw the graph corresponding to a two-layer dense neural network for regression with p input variables $x_1, \ldots, x_p$, one hidden layer with U units, activation function $h : \mathbb{R} \mapsto \mathbb{R}$ in the layer, and output $\widehat{y} \in \mathbb{R}$. How many parameters does the model have (including offsets)?

10.2 **Vectorization over units**

Mathematically, the model above can be described as

$$q_i = h\left(b_i^{(1)} + \sum_{j=1}^{p} W_{ij}^{(1)} x_j\right), \qquad i = 1, \ldots, U \tag{10.1a}$$

$$\widehat{y} = b^{(2)} + \sum_{\ell=1}^{U} W_{\ell}^{(2)} q_\ell \tag{10.1b}$$

where each node contains a linear regression model, and where each node in the hidden layer is squeezed through an activation function h. When we implement this in code, we prefer to use a *vectorized* version of these equations since it runs faster than loops and explicit sums. Vectorize the equations in (10.1) by introducing the variables

$$\mathbf{x} = \begin{bmatrix} x_1 \\ \vdots \\ x_p \end{bmatrix}, \quad \mathbf{q} = \begin{bmatrix} q_1 \\ \vdots \\ q_U \end{bmatrix}, \quad \mathbf{b}^{(1)} = \begin{bmatrix} b_1^{(1)} \\ \vdots \\ b_U^{(1)} \end{bmatrix}, \quad \mathbf{W}^{(1)} = \begin{bmatrix} W_{11}^{(1)} & \ldots & W_{1p}^{(1)} \\ \vdots & & \vdots \\ W_{U1}^{(1)} & \ldots & W_{Up}^{(1)} \end{bmatrix}, \quad \mathbf{b}^{(2)} = \begin{bmatrix} b^{(2)} \end{bmatrix}, \quad \mathbf{W}^{(2)} = \begin{bmatrix} W_1^{(2)} & \ldots & W_U^{(2)} \end{bmatrix},$$

i.e. write (10.1) on a matrix form

$$\widehat{y} = f(\mathbf{x}), \tag{10.2}$$

which does not include any explicit summation $\sum$ or looping over $\ell = 1, \ldots, U$.

10.3 **Vectorization over data points**

When processing many data points $\{\mathbf{x}_i\}_{i=1}^n$, we have the relation (10.2) for each data point $i = 1, \ldots, n$

$$\widehat{\mathbf{y}}_i = f(\mathbf{x}_i), \qquad i = 1, \ldots, n. \tag{10.3}$$

Vectorize the equations in (10.3) by introducing the variables

$$\mathbf{X} = \begin{bmatrix} \mathbf{x}_1^\mathsf{T} \\ \vdots \\ \mathbf{x}_n^\mathsf{T} \end{bmatrix}, \quad \widehat{\mathbf{Y}} = \begin{bmatrix} \widehat{\mathbf{y}}_1^\mathsf{T} \\ \vdots \\ \widehat{\mathbf{y}}_n^\mathsf{T} \end{bmatrix}, \quad \text{and} \quad \mathbf{Q} = \begin{bmatrix} \mathbf{q}_1^\mathsf{T} \\ \vdots \\ \mathbf{q}_n^\mathsf{T} \end{bmatrix}, \tag{10.4}$$

i.e. write (10.3) on a matrix form

$$\widehat{\mathbf{Y}} = f(\mathbf{X}), \tag{10.5}$$

which does not include any explicit looping $i = 1, \ldots, n$.

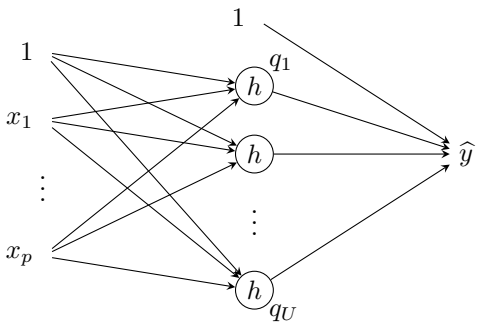
10.4 **Linear activation function**

Show that the model in the (10.1) reduces to a linear regression model if $h(x) = x$. Specifically, show how the parameters of the entire neural network relate to the parameters of the single linear regression model

$$\widehat{y} = \theta_0 + \sum_{j=1}^{p} \theta_j x_j.$$

Solutions

10.1 A dense neural network for regression with one hidden layer can be illustrated as



Each link represents a multiplication of its incoming unit with a parameter. The parameters are different for each link. The number of links in the graph (=number of parameters in the model) is consequently $(p+1) \cdot U + 1 + U$.

10.2 By stacking the equations in (10.1a) as rows in vectors we get

$$\begin{bmatrix} q_1 \\ \vdots \\ q_U \end{bmatrix} = \begin{bmatrix} h\left(b_1^{(1)} + \sum_{j=1}^{p} W_{1j}^{(1)} x_j\right) \\ \vdots \\ h\left(b_U^{(1)} + \sum_{j=1}^{p} W_{Uj}^{(1)} x_j\right) \end{bmatrix} \tag{10.6a}$$

$$\widehat{y} = b^{(2)} + \sum_{i=1}^{U} W_i^{(2)} q_i \tag{10.6b}$$

Further, by replacing the summations with matrix-vector multiplications we can write this as

$$\underbrace{\begin{bmatrix} q_1 \\ \vdots \\ q_U \end{bmatrix}}_{\mathbf{q}} = h\left(\underbrace{\begin{bmatrix} W_{11}^{(1)} & \dots & W_{1p}^{(1)} \\ \vdots & & \vdots \\ W_{U1}^{(1)} & \dots & W_{Up}^{(1)} \end{bmatrix}}_{\mathbf{W}^{(1)}} \underbrace{\begin{bmatrix} x_1 \\ \vdots \\ x_p \end{bmatrix}}_{\mathbf{x}} + \underbrace{\begin{bmatrix} b_1^{(1)} \\ \vdots \\ b_U^{(1)} \end{bmatrix}}_{\mathbf{b}^{(1)}}\right), \tag{10.7a}$$

$$\widehat{y} = \underbrace{\begin{bmatrix} W_1^{(2)} & \dots & W_U^{(2)} \end{bmatrix}}_{\mathbf{W}^{(2)}} \underbrace{\begin{bmatrix} q_1 \\ \vdots \\ q_U \end{bmatrix}}_{\mathbf{q}} + \underbrace{\begin{bmatrix} b^{(2)} \end{bmatrix}}_{\mathbf{b}^{(2)}}. \tag{10.7b}$$

By identifying all the matrices and vectors, we can in a more compact and vectorized manner write these equations as

$$\mathbf{q} = h\left(\mathbf{W}^{(1)}\mathbf{x} + \mathbf{b}^{(1)}\right), \tag{10.8a}$$

$$\widehat{y} = \mathbf{W}^{(2)}\mathbf{q} + \mathbf{b}^{(2)} \tag{10.8b}$$

Note that the activation function h acts element-wise here.

10.3 Following the solutions from the previous exercise, the equations we are about to vecotrize are

$$\mathbf{q}_i = h\left(\mathbf{W}^{(1)}\mathbf{x}_i + \mathbf{b}^{(1)}\right), \qquad i = 1, \dots, n, \tag{10.9a}$$

$$\widehat{\mathbf{y}}_i = \mathbf{W}^{(2)}\mathbf{q}_i + \mathbf{b}^{(2)}, \qquad i = 1, \dots, n. \tag{10.9b}$$

We start by taking the transpose of these equations

$$\mathbf{q}_i^\mathsf{T} = h\left(\mathbf{x}_i^\mathsf{T}\mathbf{W}^{(1)\mathsf{T}} + \mathbf{b}^{(1)\mathsf{T}}\right), \qquad i = 1, \ldots, n, \tag{10.10a}$$

$$\widehat{\mathbf{y}}_i^\mathsf{T} = \mathbf{q}_i^\mathsf{T}\mathbf{W}^{(2)\mathsf{T}} + \mathbf{b}^{(2)\mathsf{T}}, \qquad i = 1, \ldots, n. \tag{10.10b}$$

We continue by stacking these equations in rows

$$\begin{bmatrix} \mathbf{q}_1^\mathsf{T} \\ \vdots \\ \mathbf{q}_n^\mathsf{T} \end{bmatrix} = \begin{bmatrix} h\left(\mathbf{x}_1^\mathsf{T}\mathbf{W}^{(1)\mathsf{T}} + \mathbf{b}^{(1)\mathsf{T}}\right) \\ \vdots \\ h\left(\mathbf{x}_n^\mathsf{T}\mathbf{W}^{(1)\mathsf{T}} + \mathbf{b}^{(1)\mathsf{T}}\right) \end{bmatrix}, \tag{10.11a}$$

$$\begin{bmatrix} \widehat{\mathbf{y}}_1^\mathsf{T} \\ \vdots \\ \widehat{\mathbf{y}}_n^\mathsf{T} \end{bmatrix} = \begin{bmatrix} \mathbf{q}_1^\mathsf{T}\mathbf{W}^{(2)\mathsf{T}} + \mathbf{b}^{(2)\mathsf{T}} \\ \vdots \\ \mathbf{q}_n^\mathsf{T}\mathbf{W}^{(2)\mathsf{T}} + \mathbf{b}^{(2)\mathsf{T}} \end{bmatrix}. \tag{10.11b}$$

By bringing the weight matrices and offset vectors outside we get

$$\begin{bmatrix} \mathbf{q}_1^\mathsf{T} \\ \vdots \\ \mathbf{q}_n^\mathsf{T} \end{bmatrix} = h\left(\begin{bmatrix} \mathbf{x}_1^\mathsf{T} \\ \vdots \\ \mathbf{x}_n^\mathsf{T} \end{bmatrix} \mathbf{W}^{(1)\mathsf{T}} + \mathbf{b}^{(1)\mathsf{T}}\right), \tag{10.12a}$$

$$\begin{bmatrix} \widehat{\mathbf{y}}_1^\mathsf{T} \\ \vdots \\ \widehat{\mathbf{y}}_n^\mathsf{T} \end{bmatrix} = \begin{bmatrix} \mathbf{q}_1^\mathsf{T} \\ \vdots \\ \mathbf{q}_n^\mathsf{T} \end{bmatrix} \mathbf{W}^{(2)\mathsf{T}} + \mathbf{b}^{(2)\mathsf{T}}. \tag{10.12b}$$

Note that in these equations we add $+\mathbf{b}^{(1)\mathsf{T}}$ and $+\mathbf{b}^{(2)\mathsf{T}}$ to each row. In python-language we call that *broadcasting* and is heavily used in deep learning implementations.

Now we can identify the matrices in the problem description and write these equations as

$$\mathbf{Q} = h\left(\mathbf{X}\mathbf{W}^{(1)\mathsf{T}} + \mathbf{b}^{(1)\mathsf{T}}\right), \tag{10.13a}$$

$$\widehat{\mathbf{Y}} = \mathbf{Q}\mathbf{W}^{(2)\mathsf{T}} + \mathbf{b}^{(2)\mathsf{T}}, \tag{10.13b}$$

which now is in a format suitable for efficient vectorized implementation in code. For the implementation you might want to consider using the transposed version of $\mathbf{W}^{(1)}$, $\mathbf{W}^{(2)}$, $\mathbf{b}^{(1)}$ and $\mathbf{b}^{(2)}$ as your weight matrices and offset vectors to avoid taking transpose in each layer.

10.4 Alternative 1

The mathematical model corresponding to the neural network above is

$$q_i = h\left(b_i^{(1)} + \sum_{j=1}^{p} W_{ij}^{(1)} x_j\right), \qquad i = 1, \ldots, U$$

$$\widehat{y} = b^{(2)} + \sum_{\ell=1}^{U} W_\ell^{(2)} q_\ell$$

If we consider a linear activation function $h(x) = x$ in (10.1) we get

$$\begin{aligned} \widehat{y} &= b^{(2)} + \sum_{\ell=1}^{U} W_\ell^{(2)} \left(b_\ell^{(1)} + \sum_{j=1}^{p} W_{\ell j}^{(1)} x_j\right) \\ &= b^{(2)} + \sum_{\ell=1}^{U} W_\ell^{(2)} b_\ell^{(1)} + \sum_{j=1}^{p}\sum_{\ell=1}^{U} W_\ell^{(2)} W_{\ell j}^{(1)} x_j, \end{aligned}$$

which is a linear regression model

$$\widehat{y} = \theta_0 + \sum_{j=1}^{p} \theta_j x_j$$

where

$$\theta_0 = b^{(2)} + \sum_{\ell=1}^{U} W_\ell^{(2)} b_\ell^{(1)} \qquad \text{and} \qquad \theta_j = \sum_{\ell=1}^{U} W_\ell^{(2)} W_{\ell j}^{(1)}. \tag{10.14}$$

Alternative 2

We can also solve the exercise by starting from the vectorized version of the model in (10.2). With $h(x) = x$ we get

$$\mathbf{q} = \mathbf{W}^{(1)}\mathbf{x} + \mathbf{b}^{(1)}, \tag{10.15a}$$
$$\widehat{y} = \mathbf{W}^{(2)}\mathbf{q} + \mathbf{b}^{(2)} \tag{10.15b}$$

which gives us

$$\widehat{y} = \mathbf{W}^{(2)}\mathbf{W}^{(1)}\mathbf{x} + \mathbf{W}^{(2)}\mathbf{b}^{(1)} + \mathbf{b}^{(2)} \tag{10.16}$$

This can be compared with the linear regression model

$$\begin{aligned} \widehat{y} &= \theta_0 + \sum_{j=1}^{p} \theta_j x_j \\ &= \boldsymbol{\theta}_{-0}^{\mathsf{T}}\mathbf{x} + \theta_0 \end{aligned} \tag{10.17}$$

where $\boldsymbol{\theta}_{-0} = [\theta_1, \ , \ldots, \ \theta_p]^{\mathsf{T}}$. By comparing (10.16) and (10.17) we get that

$$\boldsymbol{\theta}_{-0} = \mathbf{W}^{(1)\mathsf{T}}\mathbf{W}^{(2)\mathsf{T}}, \qquad \text{and} \qquad \theta_0 = \mathbf{W}^{(2)}\mathbf{b}^{(1)} + \mathbf{b}^{(2)} \tag{10.18}$$

and by using the definitions in Exercise 10.2 this is

$$\begin{bmatrix} \theta_1 \\ \vdots \\ \theta_p \end{bmatrix} = \begin{bmatrix} W_{11}^{(1)} & \ldots & W_{U1}^{(1)} \\ \vdots & & \vdots \\ W_{1p}^{(1)} & \ldots & W_{Up}^{(1)} \end{bmatrix} \begin{bmatrix} W_1^{(2)} \\ \vdots \\ W_U^{(2)} \end{bmatrix}, \qquad \text{and} \qquad \theta_0 = \begin{bmatrix} W_1^{(2)} & \ldots & W_U^{(2)} \end{bmatrix} \begin{bmatrix} b_1^{(1)} \\ \vdots \\ b_U^{(1)} \end{bmatrix} + b^{(2)} \tag{10.19}$$

which is equivalent with the solution in (10.14).